



Professional Vue.js Development

Vue 3 · Composition API · Vite · Pinia · Vitest

Version 2.2 · © 2026 Chris Minnick · WatzThis?

DAY 1

Introduction and Vue Quickstart

Who we are, what Vue is, and your first Vite project

DAY 1

Introduction

Objectives

- Who am I?
- Who are you?
- Daily schedule
- Course schedule and syllabus

About Me

- **Chris Minnick** — author, trainer, developer
- Teaching programming since 2000
- Teaching AI since 2023
- Author of over 25 books and numerous courses on programming and AI
 - *JavaScript All-in-One For Dummies, ReactJS Foundations, Coding with AI For Dummies*, and more
- Founder of WatzThis? — books, courseware and training

Introductions

- What's your name?
- What do you do?
- JavaScript level: beginner, intermediate, or advanced?
- What do you want to know at the end of this course?
- What do you like to do for fun?
- Favorite food?

Daily Schedule (CT)

Morning

- 8:30 – 10:30 session
- 10:30 – 10:45 break
- 10:45 – 12:00 session
- 12:00 – 1:00 lunch

Afternoon

- 1:00 – 2:00 session
- 2:00 – 2:15 break
- 2:15 – 3:15 session
- 3:15 – 3:30 break
- 3:30 – 4:30 session

The Big Picture

Topics covered in this course

- Modern JavaScript, the DOM, and async/await
- Node.js, npm, Vite and build tooling
- Vue templates, directives, forms and styling
- Composition API, reactivity and lifecycle hooks
- Components, props, slots and composables
- Pinia state management, Vue Router and APIs
- Debugging with browser and Vue DevTools
- ESLint, TDD, Vitest and Vue Test Utils
- Transitions and animations

What's New in JavaScript (aka ECMAScript)?

ES2015 (ES6) — the big one

- Arrow functions
- Classes
- `let` and `const`
- Iterators and generators
- Modules (`import` / `export`)
- Promises

A new version every year

- **ES2020** `?.` and `??`
- **ES2021** `replaceAll`, logical assignment
- **ES2022** top-level `await`, class fields, `Array.at`
- **ES2023** `toSorted`, `findLast`
- **ES2024** `Object.groupBy`, `Promise.withResolvers`
- **ES2025** iterator helpers, new `Set` methods

What is Vue.js?

A **progressive, incrementally adoptable, reactive** front-end JavaScript framework.

- **Progressive** — start with a `<script>` tag on one page; grow into a full single-page app with routing, state and a build step
- **Incrementally adoptable** — use as much or as little as you need
- **Reactive** — change the data, and Vue updates the page for you
- **Framework** — opinionated about templates and components, flexible about everything else

What is Reactivity?

The way an application changes the view in response to changes in its state.

```
const count = ref(0)           // state  
count.value++                  // change the state...
```

```
<p>{{ count }}</p>           <!-- ...and the view updates itself -->
```

- You never write "find the element and update its text"
- Vue tracks which parts of the page read which state
- When state changes, only the affected parts re-render

History of Vue.js

- Created by **Evan You** after working at Google with AngularJS
 - *"I figured, what if I could just extract the part that I really liked about Angular and build something really lightweight."*
- Vue was first released in **February 2014**
- Vue 3 shipped in **September 2020** and became the default in **2022**
- Vue 2 reached **end of life on 31 December 2023**
- Vue 3.6, with the opt-in **Vapor Mode** compiler, is in release-candidate testing (fall 2026)

What Makes Vue Different?

- **Incrementally adoptable**
 - Use it with just a `<script>` include, or with modern tooling
- **Combines the best features of React and Angular**
 - More built in than React (templates, transitions, scoped CSS)
 - Less complex than Angular
- **Small:** about 34 KB gzipped for the runtime plus reactivity
 - Unused features are tree-shaken out of your bundle
- **Independent:** not created by a giant corporation
 - Governed by an independent core team, funded by its users and sponsors

Comparison of Framework File Sizes

Framework (historical versions)	Size
Angular 2 + Rx	766 KB
Angular 2	566 KB
Ember 1.13.8	486 KB
Ember 2.2.0	435 KB
Angular 1.4.5	143 KB
React 0.14.5 + React DOM + Redux	139 KB
React 0.14.5 + React DOM	133 KB
React 16.2.0 + React DOM	97.5 KB
Vue 2.4.2	58.8 KB
Inferno 1.2.2	48 KB
Preact 7.2.0	16 KB

Historical comparison of minified builds from the Vue 2 era. Current Vue 3: about 34 KB gzipped (runtime + reactivity).

Vue vs. React and Angular

All three now favour function-based, composable components: Vue's **Composition API**, React's **hooks**, Angular's **signals**.

Vue and Angular

- **Two-way data binding** with **v-model**
 - Data changes update the view
 - Input in the view updates the data
- **Built-in directives**
 - **v-for**, **v-if**, **v-on**, **v-bind**, **v-model**
- HTML-based templates

React

- **One-way data flow**
 - Events call functions that update state
 - When state changes, the component re-renders
- **Mostly plain JavaScript**
 - JSX; **map()** instead of **v-for**
- Everything is a function

Vue.js Hello World

Files: demos/hello-world.html

```
<script src="https://unpkg.com/vue@3.5.43/dist/vue.global.js"></script>
<div id="app"></div>
<script>
  const { createApp, ref } = Vue
  createApp({
    template: '<div>{{ message }}</div>',
    setup() {
      const message = ref('Hello Vue!')
      return { message }
    }
  }).mount('#app')
</script>
```

- No build step: open the file in a browser
- Vue 2 wrote `new Vue({ el: '#app', data: { ... } })`

What Makes Vue's Virtual DOM Different?

- **Compiler-informed:** templates are compiled ahead of time, so Vue knows which parts can never change
 - Static content is hoisted and skipped during diffing
 - Dynamic nodes are flagged so the diff checks only those
- **Tracks dependencies automatically**
 - Knows exactly which components need to re-render
 - Avoids re-rendering components whose state hasn't changed
- Can be rendered at runtime or precompiled at build time

Vue Versions

- **1.0** — improved template syntax, **v-for**, hot reloading, scoped CSS
- **2.0** — virtual DOM, JSX support, server-side rendering
- **3.0** — Composition API, Teleport, Fragments, Proxy reactivity
- **3.2** — **<script setup>**, **v-memo**, custom-element build
- **3.3** — generic components, better TypeScript for **defineProps**
- **3.4** — faster parser, stable **defineModel**
- **3.5** — reactive props destructure, lazy hydration, lower memory use
- **3.6** — Vapor Mode (release candidate as of fall 2026)

What Vue 3 Changed

- **Reactivity rebuilt on Proxy**
 - Added and deleted properties are now tracked; `Vue.set` is gone
- The **Composition API**, and `<script setup>` on top of it
- **Fragments**: a component may have more than one root element
- **Teleport**: render part of a component elsewhere in the DOM (modals!)
- **Multiple app instances**, each isolated — there is no global `Vue`
- First-class **TypeScript** support
- **Removed**: filters, `$on` / `$off` / `$once`, old functional component syntax

DAY 1

Vue QuickStart

Objectives

- Scaffold a project with `create-vue`
- Understand what Vite gives you
- Run, build and test a Vue app

Code Editors and IDEs

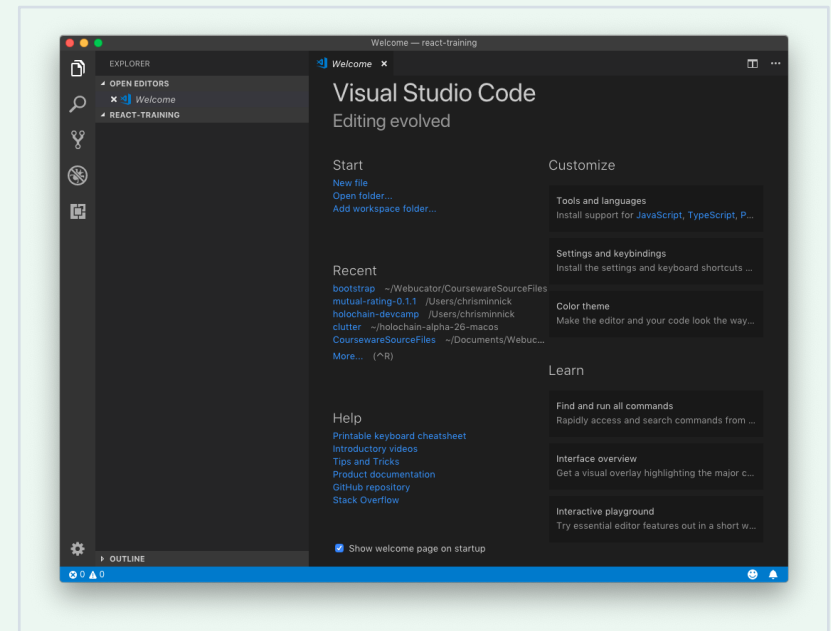
- **Visual Studio Code** with the **Vue - Official** extension (used in this course)
- JetBrains **WebStorm** / IntelliJ IDEA
- **Zed**
- **Sublime Text**
- **Neovim** / **Emacs**

Any editor with a Volar (Vue language server) client will give you Vue syntax highlighting, type checking and go-to-definition.

Installing and Configuring VS Code

Install VS Code and the extensions we use all week.

- Install Visual Studio Code
- Install the **Vue - Official** and **ESLint** extensions
- Disable Vetur if you have it
- Create a **vue-training** folder and open it in VS Code



Vite

- The build tool behind the current Vue tooling (created by Evan You)
- **Development:** serves your source as native ES modules
 - No bundling step, so startup and reloads stay fast as the app grows
 - Hot module replacement (HMR) that keeps component state
- **Production:** bundles, tree-shakes and minifies with **Rolldown**
 - Vite 8 (March 2026) replaced esbuild + Rollup with Rolldown and Oxc, written in Rust
- Framework-agnostic: Vue support comes from [@vitejs/plugin-vue](#)
- Vue CLI, which used webpack, reached end of life at the end of 2023

Using create-vue

- Scaffold a project — nothing to install globally

```
npm create vue@latest conduit
```

- Then, in the new directory:

```
npm install  
npm run dev      # start the dev server on :5173  
npm run build    # production build into dist/  
npm run preview  # serve the production build
```

- The prompts let you add TypeScript, Router, Pinia, Vitest, ESLint and Prettier up front

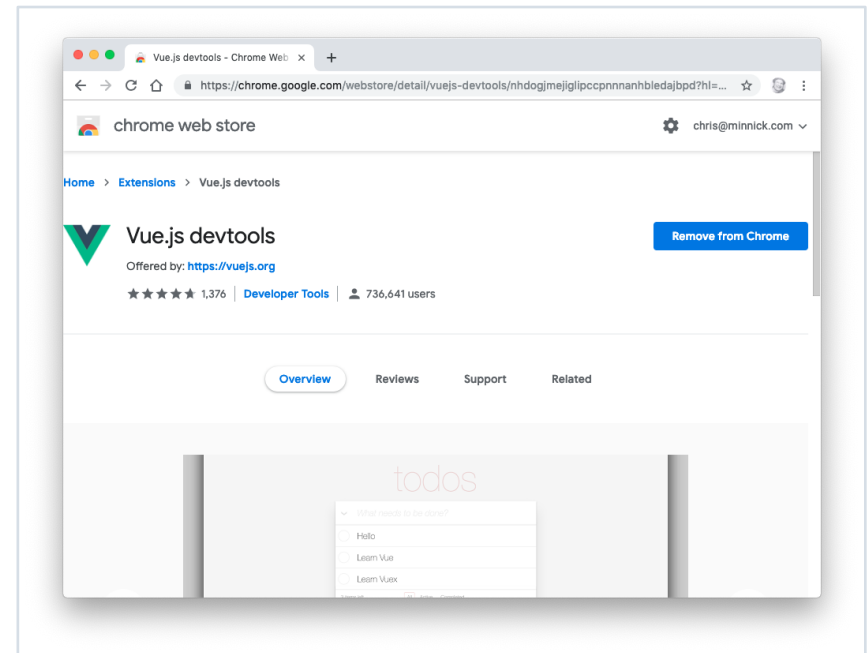
Get Started with Vite

Scaffold the Conduit project with create-vue and see hot module replacement work.

- Run `npm create vue@latest conduit`
- Say **Yes** to Vitest, Linter and Prettier only
- `npm install`, then `npm run dev`
- Open `http://localhost:5173` and edit `App.vue` — watch HMR
- Find `index.html` (in the project root) and `vite.config.js`

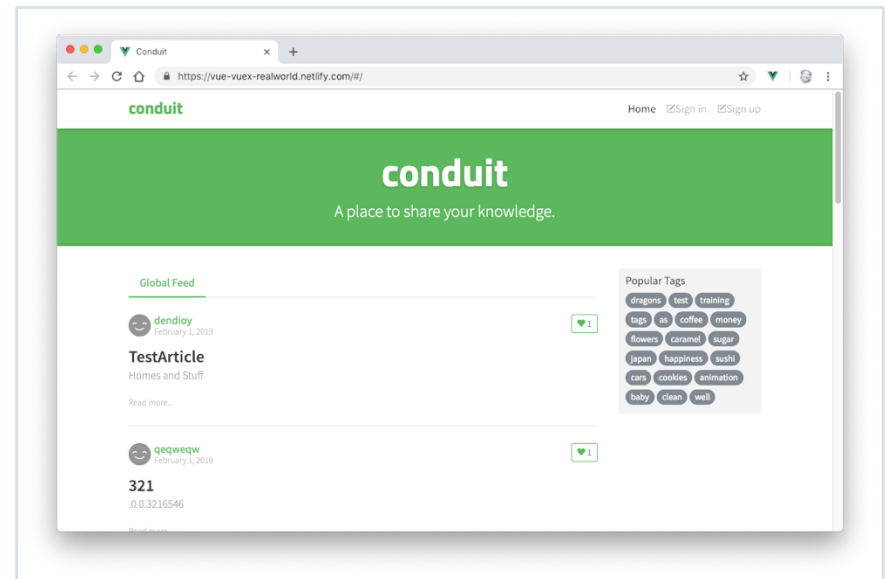
Installing and Using the Vue DevTools

- Browser extension for Chrome, Edge and Firefox
 - devtools.vuejs.org
- Or in-page, with no extension:
vite-plugin-vue-devtools
- Inspect the component tree, props and state
- Watch Pinia stores and router state
- Timeline of events and performance



Our Real-World Project: Conduit

- An open-source **Medium.com clone** — the RealWorld example app
- We'll build it step by step with:
 - Components, props, slots and events
 - A real REST API
 - Authentication
 - Routing and Pinia state management
 - Unit tests



Single-File Components

```
<script setup>
import { ref } from 'vue'
const count = ref(0)
</script>

<template>
  <button @click="count++">Clicked {{ count }} times</button>
</template>

<style scoped>
button { font-weight: bold; }
</style>
```

- **<script setup>** — the logic, using the Composition API
- **<template>** — the markup
- **<style scoped>** — CSS that applies to this component only
- One file, one component, named after the file. Compiled at build time, so the format costs nothing at runtime.

Your First Component

Create TheHeader.vue, a static Bootstrap navbar, and use it in App.vue.

- Components are reusable modules composed into user interfaces
- Vue components are **single-file components**: template, script and style
- We write them with `<script setup>`
 - Every top-level binding is available to the template
 - Importing a child component is its registration
- Add Bootstrap 5 to `index.html`

Create More Components

Build static HomeContent and TheFooter components so App.vue is composed of three parts.

- Replace the welcome components in `App.vue` with `<TheHeader />`, `<HomeContent />` and `<TheFooter />`
- Write the static HTML for the home page from the screenshot
 - banner, global feed, popular tags sidebar
- Write the static footer
- Try it yourself before looking at the solution in the lab guide

Testing Vue

Write your first component tests with Vitest and Vue Test Utils.

- Create `tests/components/Header.spec.js`
- `mount()` the component and assert that it exists
- Check the `test` section in `vite.config.js` (jsdom environment)
- Copy the spec for HomeContent and TheFooter
- Run `npm run test`, then try watch mode

DAY 1

Node.js

Objectives

- Install Node
- Test Node and get started using npm
- Understand why front-end tooling runs on Node

What is Node.js?

- A JavaScript runtime outside the browser
- Built on Google Chrome's **V8** JavaScript engine
- Open source
- **Event-driven** and **non-blocking**: it never sits and waits for I/O
- **Single-threaded**, yet handles thousands of concurrent connections with little overhead
- Used on the server, and for development automation
 - Every tool in this course — Vite, ESLint, Vitest, npm — is a Node program

How is Node.js Different?

- **JavaScript is single-threaded**
 - A Node application has one event loop, and gets the benefit of concurrency through asynchronous operations
 - Can handle thousands of concurrent connections with minimal overhead
- **No buffering**
 - Node streams data in chunks instead of loading it all into memory
- **Non-blocking**
 - Node doesn't wait for data to be returned from APIs
 - It's possible to write blocking code in Node.js (`readFileSync`), but it's discouraged in servers

The Event Loop

- **Problem:** JavaScript is single-threaded
- The event loop lets Node.js perform non-blocking I/O
 - Offloads operations to the system kernel when possible
- Node starts the event loop when it starts your program
- Each turn of the loop runs through **six phases**
- Between turns, Node checks whether anything is still pending, and exits if nothing is

Recap: How is Node.js Different?

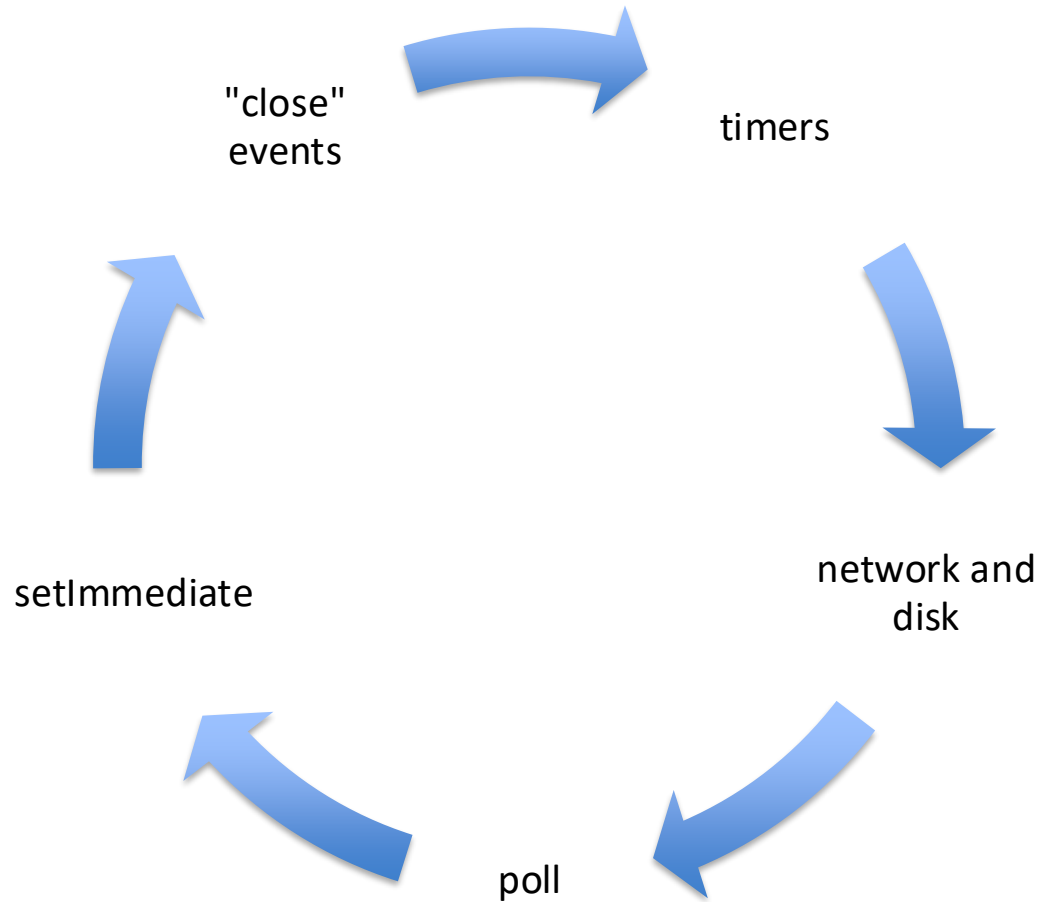
- **JavaScript is single-threaded**
 - A Node application has one event loop, and gets the benefit of concurrency through asynchronous operations
 - Can handle thousands of concurrent connections with minimal overhead
- **No buffering**
 - Node streams data in chunks instead of loading it all into memory
- **Non-blocking**
 - Node doesn't wait for data to be returned from APIs
 - It's possible to write blocking code in Node.js (`readFileSync`), but it's discouraged in servers

Event Loop Phases

1. **timers** — runs `setTimeout()` and `setInterval()` callbacks
2. **pending callbacks** — I/O callbacks deferred from the previous turn
3. **idle, prepare** — internal use only
4. **poll** — retrieve new I/O events; run their callbacks
5. **check** — run `setImmediate()` callbacks
6. **close callbacks** — e.g. `socket.on('close', ...)`

Promise callbacks (microtasks) run between each callback, before the loop moves on.

Event Loop



EventEmitter

- Node is event driven
- **EventEmitter** is a class that lets you listen for named events and run code when they occur
- A very loose way of coupling different parts of your application

```
import { EventEmitter } from 'node:events'

const ee = new EventEmitter()

ee.on('someEvent', (who) => {
  console.log(`event has occurred: ${who}`)
})

ee.emit('someEvent', 'Ada')
```

A Simple Node.js Example

```
import http from 'node:http'

const hostname = '127.0.0.1'
const port = 3000

const server = http.createServer((req, res) => {
  res.statusCode = 200
  res.setHeader('Content-Type', 'text/plain')
  res.end('Hello World\n')
})

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`)
})
```


Modules Overview

- Node originally used **CommonJS** for loading modules
 - Original name was ServerJS
 - A way to import dependencies outside the browser
- `module.exports` — encapsulates code into a module
- `require()` — imports a module into JavaScript code
- Today Node also supports standard **ES Modules** (`import` / `export`), which is what Vite projects use

CommonJS Example

circle.js

```
exports.area = function (r) {  
  return Math.PI * r * r  
}
```

foo.js

```
const circle = require('./circle.js')  
console.log('Area of circle: ' + circle.area(4))
```

ES Modules: import and export

JavaScript's standard module format. This example exports and imports a function by name.

circle.js

```
export function area(r) {  
  return Math.PI * r * r  
}
```

foo.js

```
import { area } from './circle.js'  
console.log('Area of circle: ' + area(4))
```

ES Modules in Node.js

- Node.js supports both CommonJS and ES Modules
- To run the previous example, add to `package.json`:

```
{ "type": "module" }
```

- Node now treats `.js` files in this package as ES Modules
 - Run the entry file with `node foo.js`
- Alternative: use `.mjs` files (and matching imports)
- Use `.cjs` for CommonJS files in an ES Module package
- Include extensions in relative imports: `./circle.js`

Front-end Node

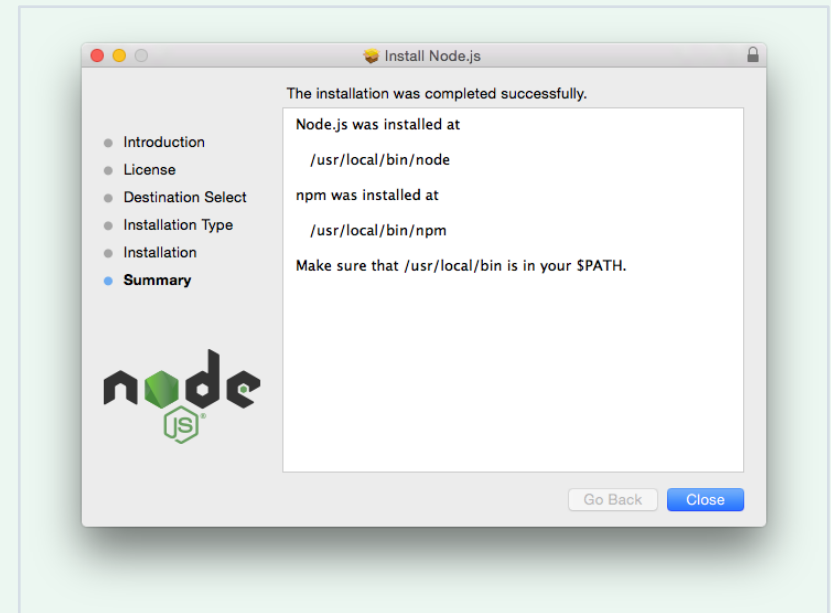
Node has become an essential part of front-end development. It runs:

- Package managers (npm, pnpm, yarn)
- Dev servers and build tools (Vite)
- Test runners (Vitest, Playwright)
- Linters and formatters (ESLint, Prettier)
- CSS tools (PostCSS, Sass)
- Headless browsers for testing (Playwright, Puppeteer)
- ...and much more

Getting Started with Node.js and npm

Explore Node in the REPL, run a script from a file, and learn the core npm commands.

- Check your version: `node -v`
- Explore the REPL: expressions, `process.version`, `process.platform`
- Write `javascript.js` and run it with `node`
- Read a name from `process.argv`
- Part 2: `npm -v`, `npm ls`, `npm outdated`, `npm help`



DAY 1

Tooling

npm, package.json, linting, transpiling and bundling

Why Have Tooling?

Front-end web development is complex. Modern tooling helps you achieve four software-development best practices:

- **Version control** — every change tracked (git)
- **Automation** — linting, testing and building run with one command
- **Reproducible builds** — the same inputs produce the same output on every machine (lockfiles)
- **Test-driven development** — fast, automatic feedback

npm

- Installs, publishes and manages Node packages
- Bundled and installed with Node
- Installs packages from the **npm registry** (npmjs.com)
- Runs project scripts: `npm run dev`, `npm test`
- `npmx` runs a package's command without installing it globally

Other Package Managers

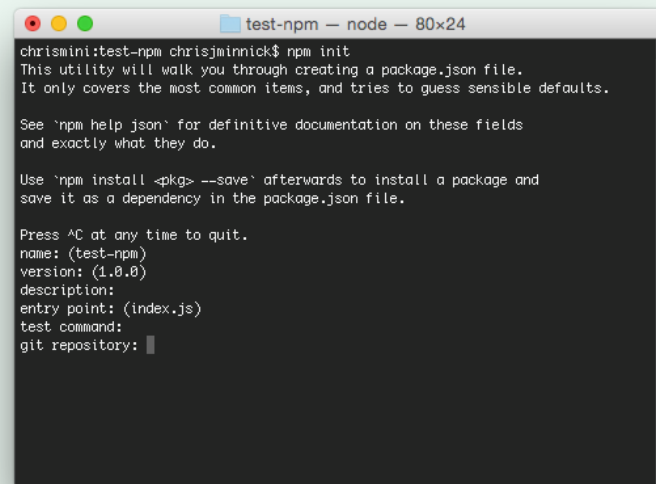
- **npm** ships with Node.js and is the default
- **yarn** — created by Facebook; offers a Plug'n'Play install mode
- **pnpm** — hard-links packages from a global store, so disk use stays flat
 - Strict by default: a package can only import what it declares
- **bun** — a JavaScript runtime with a very fast built-in installer

All four read `package.json`. They differ mainly in their lockfile and how they lay out `node_modules`.

Initialize npm

Create a package.json with npm init and automate a tiny build with npm scripts.

- `npm init` in `labs/lab07`
- Look at `package.json`; run `npm install` to create `package-lock.json`
- Run scripts with `npm run` and `npm run test`
- Add a `build` script and a `prebuild` script that runs your tests first



```
test-npm — node — 80x24
chrismini:test-npm chrisjminnick$ npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See 'npm help json' for definitive documentation on these fields
and exactly what they do.

Use 'npm install <pkg> --save' afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
name: (test-npm)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
```

node_modules

- Two ways to install npm packages: **locally** or **globally**
- Local packages go into the `node_modules` directory of your project
- Always run `npm install` from the folder that holds `package.json` — the root of your project
- `npm update` updates local packages within their allowed ranges
- `npm outdated` shows which packages have newer versions

Never commit `node_modules`. It's rebuilt from `package.json` and the lockfile.

package.json and Semantic Versioning

- Manages locally installed npm packages
- Documents the packages your project depends on, and the version **ranges** it can use
- With the lockfile, makes your build reproducible
- Versions use **semver**: **MAJOR.MINOR.PATCH** — e.g. **3.5.13**
- Range operators:
 - **~3.5.13** — patch updates only: **3.5.x**
 - **^3.5.13** — minor and patch updates: **3.x.x** (npm's default)
 - ***** — any version (avoid)

npm install

- Downloads and installs a package

```
npm install vue           # dependencies (the default)
npm install -D vitest     # devDependencies
npm install -E vue@3.5.13 # exact version, no range
npm install -g pnpm       # global, not in the project
npm ci                   # clean install from lockfile (CI)
```

- **dependencies:** needed by your app at runtime
- **devDependencies:** only needed to build and test it

package.json Fields

Every npm package contains this file. It holds metadata about the project:

Identifier metadata

- `name`
- `version`
- `license`
- `description`

Functional metadata

- `type ("module")`
- `main / exports` (entry point)
- `repository`
- `scripts`
- `dependencies`
- `devDependencies`
- `engines`

Making a Web Server

Write and run a Node HTTP server using ES module syntax.

- `npm init` in `labs/lab08`, with type `module`
- Create `server.js` using `node:http`
- Run it with `node server.js`
- Visit `http://127.0.0.1:3000`
- Change the response to return a full HTML page

Code-along: Set Up Your Own Tooling

Files: demos/Vue-Starter

How a Vue app with single-file components works, starting from an empty directory.

- Only three dependencies: `vue`, `vite`, `@vitejs/plugin-vue`

```
npm init -y  
npm install vue  
npm install -D vite @vitejs/plugin-vue
```

- Add `index.html`, `vite.config.js`, `src/main.js`, `src/App.vue`
- Add `"dev": "vite"` and `"build": "vite build"` scripts

Static Code Analysis

Objectives

- Learn about lint tools
- Use ESLint
- Configure ESLint with a flat config
- Manual testing with a local dev server

Lint Tools

- **ESLint** — the standard. Pluggable and "agenda free": it doesn't push a particular style
 - Flat config (`eslint.config.js`) since version 9
- **Biome** — linter and formatter in one, written in Rust
- **oxlint** — a very fast linter, also Rust; often run alongside ESLint
- **Prettier** — a formatter, not a linter: it's about layout, not correctness

JSLint and JSHint came first, and are now rarely used.

Configuring ESLint

Two ways:

- **Configuration comments** — embed config in a JS file

```
/* eslint eqeqeq: "off", curly: "error" */
```

- **Configuration files** — `eslint.config.js`, the "flat config" format
 - Flat config became the default in ESLint 9
 - It exports an **array**; each entry applies to the files it matches
 - `.eslintrc.*` and `.eslintignore` were removed in ESLint 10 (Feb 2026)

ESLint: What Can Be Configured?

- **languageOptions**
 - `ecmaVersion`, `sourceType`, and the parser to use
 - `globals` — the predefined variables the code can rely on
- **files / ignores**
 - Which paths this configuration object applies to
- **plugins**
 - Imported objects now, not strings resolved by name
- **rules**
 - Turn rules on at different levels, with options

ESLint Rules

- Three levels:
 - "off" or 0 — rule not applied
 - "warn" or 1 — reported, but doesn't fail the run
 - "error" or 2 — reported, and ESLint exits with an error code

```
// eslint.config.js
export default [
  {
    files: ['**/*.js'],
    rules: {
      eqeqeq: 'off',
      curly: 'error',
      quotes: ['error', 'double'],
    },
  },
]
```

Automate Linting

Install ESLint, write a flat config, and run the linter automatically before your tests.

- `npm install --save-dev eslint @eslint/js globals`
- Write an `eslint.config.js` flat config
- Create an npm script called `lint`
- Fix the errors (or tune the rules) until only warnings remain
- Make `prebuild` run `lint` before `test`

Transpilers

- Not all browsers support the same JavaScript and CSS features
 - Check support at **caniuse.com**
- Transpilers convert modern code to a form older browsers accept
- In a Vite project this happens at build time
 - Vite 8 uses **Oxc** for JavaScript and TypeScript, and **Lightning CSS** for CSS
 - The default target is "baseline widely available" browsers
 - Set **build.target** in **vite.config.js** to support older browsers

Browser Compatibility Tables

Feature-by-feature support across browsers and versions (compat-table.github.io)

Feature name	Current browser	Traceur	Babel 6 + core-js 2	Babel 7 + core-js 2	Babel 7 + core-js 3	Closure 2019.02	Type-Script + core-js 2	Type-Script + core-js 3	es7-shim	IE 11	Edge 17	Edge 18	Edge 19 Preview	FF 60 ESR	FF 64	FF 65	FF 66
Proxy internal calls, Array.prototype.includes	Yes	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
2017 features																	
Object static methods	4/4	0/4	4/4	4/4	4/4	3/4	4/4	4/4	3/4	0/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4
String padding	2/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2
trailing commas in function syntax	2/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2	0/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2
async functions	16/16	4/16	4/16	4/16	4/16	10/16	9/16	9/16	0/16	0/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16
shared memory and atomics	17/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17
2017 misc																	
Proxy "ownKeys" handler, duplicate keys for non-extensible targets (ES 2017 semantics)	Yes	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes	No	No	No	No
RegExp "u" flag, case folding	Yes	No	No	No	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes	Yes
arguments.caller removed	Yes	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
2017 annex b																	
Object.prototype getter/setter methods	16/16	0/16	16/16	16/16	16/16	0/16	16/16	16/16	0/16	8/16	14/16	14/16	14/16	16/16	16/16	16/16	16/16
Proxy internal calls, getter/setter methods	4/4	0/4	0/4	0/4	0/4	0/4	0/4	0/4	0/4	0/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4
assignments allowed in for-in head in non-strict mode	Yes	Yes	No	No	No	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
2018 features																	
object rest/spread properties	2/2	0/2	2/2	2/2	2/2	0/2	2/2	2/2	0/2	0/2	0/2	0/2	0/2	2/2	2/2	2/2	2/2
Promise.prototype.finally	3/3	0/3	3/3	3/3	3/3	3/3	3/3	3/3	0/3	0/3	0/3	3/3	3/3	3/3	3/3	3/3	3/3
s (dotAll) flag for regular expressions	Yes	?	Yes	Yes	Yes	No	?	?	No	No	No	No	No	No	No	No	No
RegExp named capture groups	Yes	No	Yes	Yes	Yes	No	No	No	No	No	No	No	No	No	No	No	No
RegExp Lookbehind Assertions	Yes	No	No	No	No	No	No	No	No	No	No	No	No	No	No	No	No
RegExp Unicode Property Escapes	Yes	No	Yes	Yes	Yes	No	No	No	No	No	No	No	No	No	No	No	No

Babel, esbuild, and Oxc

- **Babel** — the original JavaScript transpiler
 - Plugin-based; the reference implementation for proposed syntax
- **esbuild** — written in Go, roughly 100× faster than Babel
 - Vite used it for dev-time transforms through Vite 7
- **Oxc** — Rust compiler toolchain (parser, transformer, minifier)
 - What Vite 8 uses for transforms

You rarely configure any of these directly in a Vite project.

Transpiling Example

ES2015

```
let myFunc = () => {  
  let myVar = {  
    myProp,  
  }  
}
```

ES5

```
"use strict";  
  
var myFunc = function myFunc() {  
  var myVar = {  
    myProp: myProp  
  };  
};
```

Bundlers

A bundler resolves your import graph and writes deployable files.

- **webpack** — the long-time incumbent; configured with loaders and plugins
- **Rollup** — what Vite used for production builds through Vite 7
- **Rolldown** — Rust, Rollup-compatible; Vite 8's single bundler
- **esbuild** — fast Go bundler; Vite 7 used it to pre-bundle dependencies

In development, Vite doesn't bundle your source code at all.

How Vite Works

- **In development**
 - Serves your source over HTTP as native ES modules
 - Transforms each file only when the browser asks for it
 - Pre-bundles dependencies once, then caches them
- **In production (`vite build`)**
 - Bundles with Rolldown: tree-shaking, code-splitting, minification
 - Each lazily imported route becomes its own chunk
- **Plugins** extend both pipelines — `@vitejs/plugin-vue` compiles SFCs

The Document Object Model

Objectives

- Understand how the DOM works
- Select DOM nodes
- Manipulate the DOM with JavaScript
- See how libraries and frameworks change that work

What is the DOM?

- The browser's JavaScript API for HTML documents
- Represents elements as a **tree** of objects
- Objects in the tree can be found, read and changed with methods
- Change the tree, and the browser re-renders the page

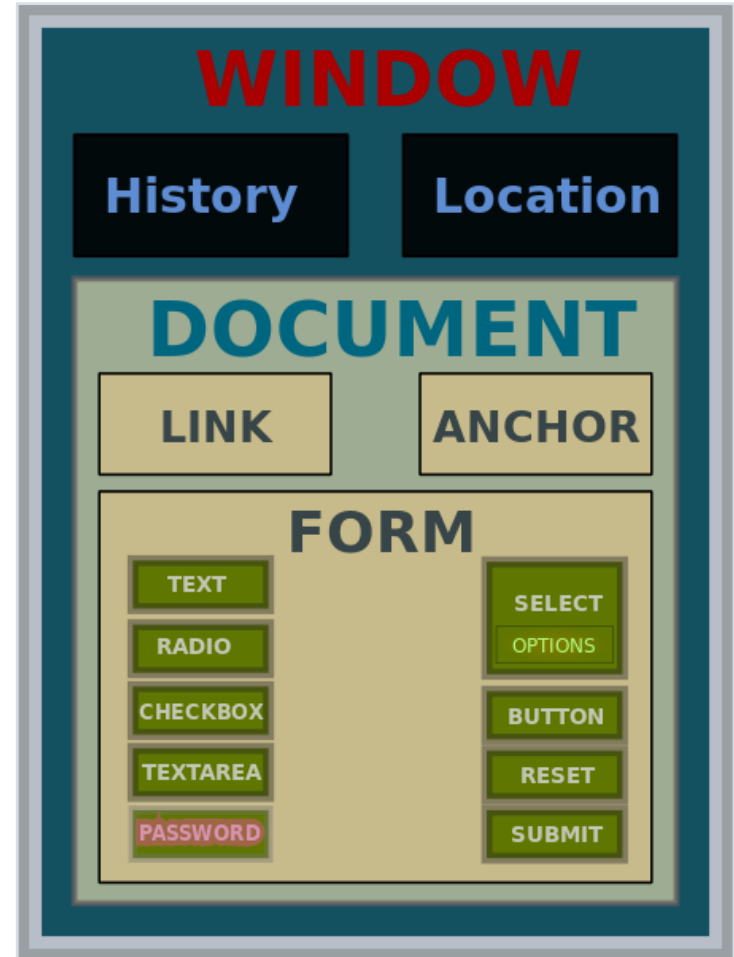


Image: JohnManuel, GFDL / CC BY-SA 3.0, via Wikimedia Commons

Understanding Nodes

DOM interfaces that inherit from **Node**:

- **Document**
- **Element**
- **CharacterData**
 - Text
 - Comment
- **DocumentType**

Every **Node** also inherits from **EventTarget**, so every node can receive events.

EventTarget

- An interface implemented by objects that can receive events (event targets)
- Examples:
 - `Element`
 - `Document`
 - `Window`
- Event targets let you register handlers:

```
button.addEventListener('click', (event) => {  
  console.log('clicked', event.target)  
})
```

DOM Events

Things that happen in the DOM. A few common ones:

Mouse and pointer

- click, dblclick
- mousedown, mouseup
- mouseenter, mouseleave
- mouseover, mouseout
- mousemove, wheel

Keyboard and input

- keydown, keyup
- beforeinput, input
- compositionstart, compositionend
- select

Focus, page and media

- focus, blur
- focusin, focusout
- load, error, abort
- resize, scroll

Other Events

- Many interfaces implement or inherit events
 - Forms: `submit`, `reset`, `change`
 - Network and storage: `online`, `offline`, `storage`
 - Media: `play`, `pause`, `ended`
 - Page lifecycle: `DOMContentLoaded`, `visibilitychange`, `pagehide`
- Full reference: **`developer.mozilla.org/en-US/docs/Web/Events`**

Element

The interface for elements in a document. It inherits properties and methods from `Node` and `EventTarget`.

Common properties

- `innerHTML`, `textContent`
- `attributes`
- `classList`
- `id`
- `tagName`

Common methods

- `querySelector()`, `querySelectorAll()`
- `addEventListener()`
- `setAttribute()`, `getAttribute()`
- `closest()`, `remove()`
- `getElementById()` (on `document`)

Manipulating HTML with the DOM

You can get and set properties of HTML elements with JavaScript through the DOM.

```
<!-- starting HTML -->  
<p id="favoriteMovie">The Matrix</p>  
  
<script>  
  document.getElementById('favoriteMovie')  
    .textContent = 'The Godfather'  
</script>  
  
<!-- the DOM, and the page, now contain -->  
<p id="favoriteMovie">The Godfather</p>
```

Manipulating HTML with the DOM

```
<ol id="favoriteSongs">
  <li class="song"></li>
  <li class="song"></li>
  <li class="song"></li>
</ol>

<script>
  const mySongs =
    document.querySelectorAll('#favoriteSongs .song')
  mySongs[0].textContent = 'My New Favorite Song'
</script>
```

Manipulating HTML with jQuery

```
<ol id="favoriteSongs">  
  <li class="song"></li>  
  <li class="song"></li>  
  <li class="song"></li>  
</ol>
```

```
<script>  
  $('#favoriteSongs .song').first()  
    .text('My New Favorite Song')  
</script>
```

Manipulating HTML with React

```
<div id="favoriteSongs"></div>

function FavoriteSongs({ songs }) {
  return (
    <ol>
      {songs.map((song) => (
        <li key={song.id} className="song">{song.title}</li>
      ))}
    </ol>
  )
}

createRoot(document.getElementById( 'favoriteSongs' ))
  .render(<FavoriteSongs songs={songs} />)
```


Manipulating HTML with Vue.js

```
<div id="favoriteSongs">
  <ol>
    <li v-for="song in songs" :key="song.title">
      {{ song.title }}
    </li>
  </ol>
</div>
```

```
import { createApp, ref } from 'vue'

createApp({
  setup() {
    const songs = ref([
      { title: 'Walk This Way' },
      { title: 'Give It Away' },
      { title: 'My Way' },
    ])
    return { songs }
  },
}).mount('#favoriteSongs')
```

DAY 1

Debugging JavaScript

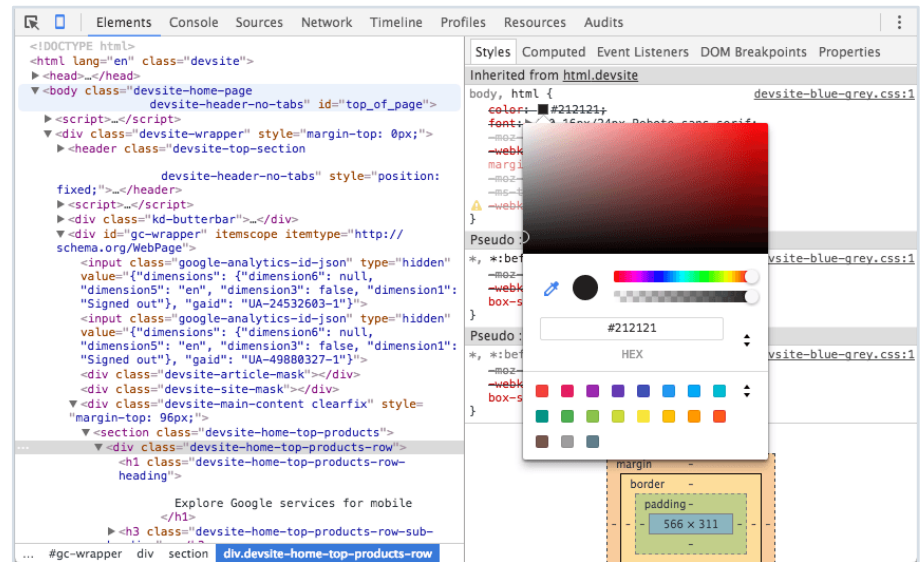
Browser DevTools and Vue DevTools

Browser Development Tools

- Every modern browser has them: Chrome, Edge, Firefox, Safari
- Open with **F12**, **Ctrl+Shift+I** (Windows/Linux) or **Cmd+Option+I** (macOS)
- The panels you'll use most:
 - **Elements** — the live DOM and CSS
 - **Console** — logs, errors, and a JavaScript REPL
 - **Sources** — breakpoints and step-through debugging
 - **Network** — every request, its timing and its response

Installing and Using the Vue DevTools

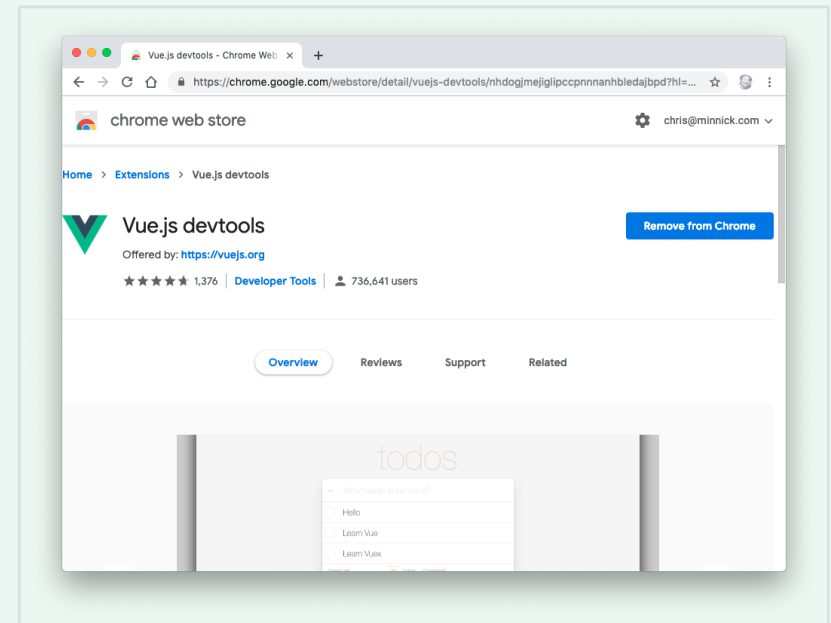
- Install from **devtools.vuejs.org**
 - Or add **vite-plugin-vue-devtools** for an in-page version
- Adds Vue panels alongside the browser's own
 - Component tree, props and state
 - Pinia stores and routes
 - Events timeline



Manual In-Browser Testing and Debugging

Get comfortable inspecting, changing and stepping through your app in the browser.

- Run the Conduit dev server
- Install Vue DevTools (extension or Vite plugin)
- Elements: edit styles live; hide an element
- Console: change `document.body.innerHTML`
- Sources: set a breakpoint in a `.vue` file and reload
- Vue tab: inspect a component's props and state



DAY 2

Getting Started With Vue.js

Core features: templates, directives, reactivity, events and lifecycle

DAY 2

Basic Vue.js Features

Composition API, <script setup>, template syntax and directives

Options API vs. Composition API

Both are fully supported. This course uses the Composition API with `<script setup>`, which is what current Vue code and documentation use.

Options API

- Code grouped by **option**: `data`, `computed`, `methods`, `watch`
- A single feature is spread across several options
- Reuse means **mixins**

Composition API

- Code grouped by **feature**
- Reuse means **composables** — ordinary functions
- Better type inference and IDE support

Creating and Mounting an App

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'

const app = createApp(App)    // an isolated application instance

app.use(createPinia())        // plugins are installed per app
app.use(router)
app.mount('#app')             // where to render
```

There's no global **Vue** object, so two apps on the same page never share plugins, components or configuration.

<script setup>

```
<script setup>
import { computed, ref } from 'vue'
import ArticleList from './ArticleList.vue'

const count = ref(0) // state
const doubled = computed(() => count.value * 2) // derived state

function increment() { // behavior
  count.value++
}
</script>
```

- Every top-level binding is available to the template — variables, functions and imports
- No **export default**, no **components** object, no **this**

ref and computed

- `ref(value)` wraps any value in a reactive container
 - Read and write it as `.value` in script
 - Vue unwraps it for you in the template
- `computed(getter)` derives a cached, read-only value
 - Re-evaluates only when something it read has changed
- `reactive(object)` makes an object reactive without `.value`
 - Objects only, and it doesn't survive destructuring

Reach for `ref` by default.

Compiler Macros

Available only at the top level of `<script setup>`. They're **not imported** — the compiler handles them.

- `defineProps({ ... })` — declare the props
- `defineEmits(['change'])` — declare the events
- `defineModel()` — a writable `v-model` ref (3.4+)
- `defineExpose({ ... })` — expose members to a parent's template ref
- `defineOptions({ ... })` — set component options such as `name`
- `withDefaults()` — default values for TypeScript-declared props

App-level Configuration

```
app.component('BaseButton', BaseButton) // global component
app.directive('focus', vFocus)          // custom directive
app.use(plugin)                           // install a plugin
app.provide('apiUrl', API_URL)            // for every descendant
app.config.errorHandler = (err) => {}     // app-wide errors
app.mount('#app')                         // render it
```

Vue 2 set all of these on the global **Vue**; Vue 3 scopes them to the app instance.

DAY 2

Vue Template Syntax

Interpolation, expressions and directives

Vue Templates

- Vue compiles templates into **render functions** that produce virtual DOM
 - In a Vite project, at build time; with the global build, in the browser
- Templates are usually written in HTML
- You can also write render functions in JSX, or in JavaScript with `h()`

```
<template>
  <h1>{{ title }}</h1>
</template>
```

Using HTML in Templates

- Mustache syntax (double curly braces) for text interpolation

```
<div>{{ customerName }}</div>
```

- Interpolation always outputs **plain text** — HTML is escaped
- To output HTML, use the **v-html** directive

```
<p>{{ info }}</p>
```

<!-- shows the tags as text -->

```
<p v-html="info"></p>
```

<!-- renders info as HTML -->

Only use **v-html** with trusted content: it's an XSS risk.

Using JavaScript in Templates

Vue supports any single JavaScript **expression** inside templates and bindings.

```
<p>{{ count + 1 }}</p>
<p>{{ ok ? 'YES' : 'NO' }}</p>
<p>{{ message.split('').reverse().join('') }}</p>
<div :id="'list-' + id"></div>
```

- Expressions only: no statements (**let**, **if**, loops)
- Keep them short — move logic into a computed property

Vue.js Directives

- Special attributes that take a JavaScript expression as their value
- Prefixed with **v-** followed by the directive name

```
<div v-if="test">The test is true</div>
```

- When the value of the expression changes, Vue updates the DOM

Directive Arguments

- Some directives take an **argument**, written after a colon

```
<button v-on:click="doSomething">Click Me</button>  
<a v-bind:href="url">Link</a>
```

- `v-on:click` — the argument is the event name
- `v-bind:href` — the argument is the attribute name
- Arguments can be dynamic: `v-bind:[attrName]="value"`

Directive Modifiers

- **Modifiers** change how a directive is bound
- They follow a `.` after the argument

```
<form v-on:submit.prevent="doSomething">...</form>
```

- `.prevent` calls `event.preventDefault()` before calling `doSomething`, so the browser doesn't submit and reload the page
- Modifiers can be chained: `@click.stop.prevent`

Vue.js Built-in Directives

Most used

- `v-bind` — bind attributes to data
- `v-if` / `v-else-if` / `v-else` — conditional rendering
- `v-for` — loop over arrays and objects
- `v-on` — event handling
- `v-model` — two-way binding for inputs

Also built in

- `v-show` — toggle `display`
- `v-html`, `v-text` — set content
- `v-slot` — named and scoped slots
- `v-once`, `v-memo` — skip re-rendering
- `v-pre`, `v-cloak` — compilation control

Custom Directives

- Components are the main way to reuse code
- Sometimes you need low-level DOM access on plain elements

```
<script setup>
// in <script setup>, vFocus is usable as v-focus
const vFocus = {
  mounted: (el) => el.focus(),
}
</script>

<template>
  <input v-focus />
</template>
```

- Hook names match the component lifecycle: Vue 2's **inserted** is now **mounted**

Loops and Lists

Use **v-for** to render a list of items from an array or object.

```
<ul>
  <li v-for="song in songs" :key="song.id">
    {{ song.title }}
  </li>
</ul>
```

```
const songs = ref([
  { id: 1, title: 'Hotel California' },
  { id: 2, title: 'April in Paris' },
])
```

Using key

- Always provide a **key** with **v-for**

```
<li v-for="song in songs" :key="song.id">
  {{ song.title }}
</li>
```

- The key gives each element a stable **identity**, so Vue can reuse, move and remove the right DOM nodes
- Use a stable, unique id — **not** the array index
- The key goes on the element that carries **v-for**, including a **<template v-for>** (Vue 2 didn't allow that)

Using v-for

- Renders a list based on an **array**, an **object**, or a **number range**

```
<li v-for="(song, index) in songs" :key="song.id">
  {{ index }}: {{ song.title }}
</li>
<li v-for="(value, key) in person" :key="key">
  {{ key }}: {{ value }}
</li>
<span v-for="n in 5" :key="n">{{ n }}</span>
```

- Often used with HTML lists, table rows, and repeated components
- **v-for** can repeat a **<template>** to output several elements per item

v-for with Arrays

Template

```
<ul id="favorite-songs">
  <li v-for="song in songs" :key="song.title">
    {{ song.title }}
  </li>
</ul>
```

State

```
const songs = ref([
  { title: 'Hotel California' },
  { title: 'April in Paris' },
])
```

Result

- Hotel California
- April in Paris

v-for with Objects

With an object, **v-for** iterates over the object's properties.

Template

```
<ul>
  <li v-for="(value, key) in person" :key="key">
    {{ key }}: {{ value }}
  </li>
</ul>
```

Result

- firstName: Joe
- lastName: Smith

State

```
const person = ref({
  firstName: 'Joe',
  lastName: 'Smith',
})
```

Conditional Rendering

Use `v-if`, `v-else-if` and `v-else`:

```
<h1 v-if="thing === true">It is so</h1>  
<h1 v-else-if="thing === undefined">We don't know</h1>  
<h1 v-else>It is not so</h1>
```

- `v-else-if` and `v-else` must immediately follow a `v-if` or `v-else-if` element
- Put `v-if` on a `<template>` to toggle several elements at once
- Avoid `v-if` and `v-for` on the same element — wrap one in a `<template>`

v-if vs. v-show

```
<p v-if="loggedIn">Welcome back!</p>
<p v-show="menuOpen">Menu</p>
```

v-if

- Adds and removes the element
- Lazy: nothing renders until true
- Child components are created and destroyed
- Higher **toggle** cost
- Works with **v-else** and **<template>**

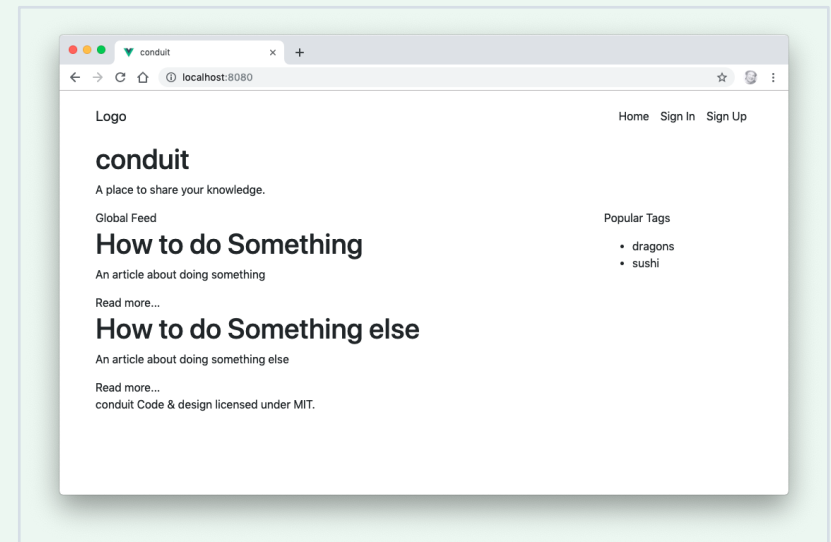
v-show

- Always renders; toggles CSS **display**
- Element and children stay alive
- Higher **initial render** cost
- Cheap to toggle
- No **v-else**; not on **<template>**

Static Vue View

Render the article list from JSON with `v-for`, `v-if` and props.

- Create **GlobalFeed**, **ArticleList** and **ArticlePreview**
- Load `articles.json` into a `ref`
- Use `v-if` for the empty state and `v-for` (with `:key`) for the list
- Pass each article to `ArticlePreview` as a prop with `defineProps`
- Add tests for the new components



Binding HTML Classes

- `:class` can take an **object** to toggle classes on and off

```
<div :class="{ active: isActive, 'text-danger': hasError }"></div>
```

- ...or an **array** to apply a list of classes

```
<div :class="[baseClass, { active: isActive }]"></div>
```

- Vue merges these with any static `class` attribute on the element

Adding Styles Conditionally

Bind a computed property or a ref to the class to change styles based on conditions.

```
<div :class="[ { active: isActive }, errorClass]"></div>
```

```
const isActive = ref(true)
const hasError = ref(false)

const errorClass = computed(() =>
  hasError.value ? 'text-danger' : ''
)
```


Binding Styles

- Inline object syntax for `:style` (camelCase or quoted kebab-case keys)

```
<div :style="{
  color: activeColor,
  fontSize: fontSize + 'px'
}"></div>
```

- Bind to a style object to keep the template cleaner

```
<div :style="styleObject"></div>
```

```
const styleObject = reactive({
  color: 'red',
  fontSize: '13px',
})
```

Outputting Raw HTML

Files: `demos/output-raw-html`

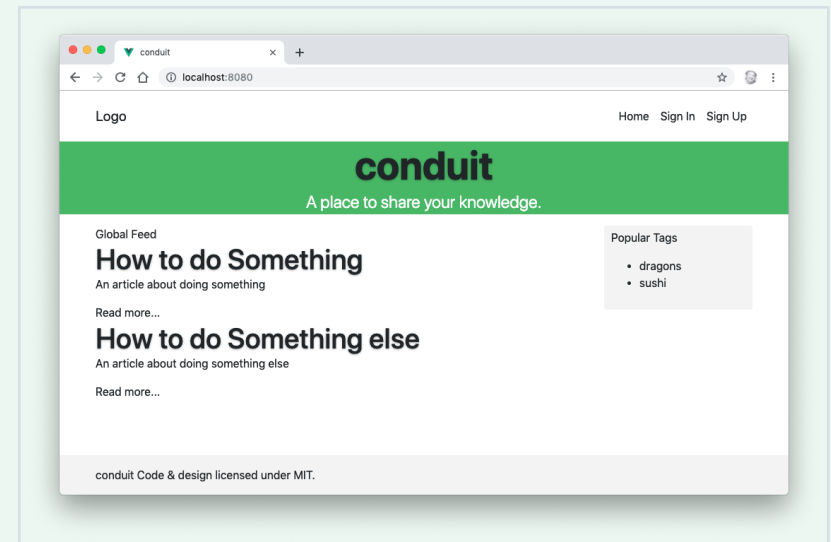
- Open `index.html` in a browser — no build step
- Compare `{{ rawHtml }}` with `v-html="rawHtml"`
- Look at the ref in `js/index.js`

`v-html` sets `innerHTML`. Never use it with user-supplied content.

Styling Vue Components

Style HomeComponent and TheFooter with scoped CSS.

- Add a `<style scoped>` block to HomeComponent
- Style the banner, sidebar and feed toggle
- Add scoped styles to TheFooter
- Inspect the result in DevTools — find the `data-v-...` attributes



Computed Properties

```
import { computed, ref } from 'vue'

const firstName = ref('Ada')
const lastName = ref('Lovelace')

const fullName = computed(() => `${firstName.value} ${lastName.value}`)
```

Computed Properties

- Useful for logic too complex to sit in a template
- Created by calling `computed()` with a getter function

```
const message = ref('Hi There')  
  
const reversedMessage = computed(() =>  
  message.value.split('').reverse().join('')  
)
```

- `computed()` returns a read-only **ref**
 - `reversedMessage.value` in script
 - `{{ reversedMessage }}` in the template

Benefits of Computed Properties

- **Cached:** unlike a function call in the template, a computed property doesn't re-run unless a reactive value it read has changed
- **Declarative:** often a cleaner alternative to a watcher
- **Readable and testable:** the logic has a name

A getter should have no side effects. Don't mutate state, make requests or touch the DOM inside one.

Filtering and Sorting a List with a Computed Property

Files: `demos/filter-with-computed-property`

- Open `index.html` in a browser
- Type in the box to filter by name or age
- Click a column heading to sort; click again to reverse
- Read `js/index.js`: one computed property, `filterIt`, does both

Using Setters in Computed Properties

- Computed properties are getter-only by default
- Pass an object with **get** and **set** when you need to write to one

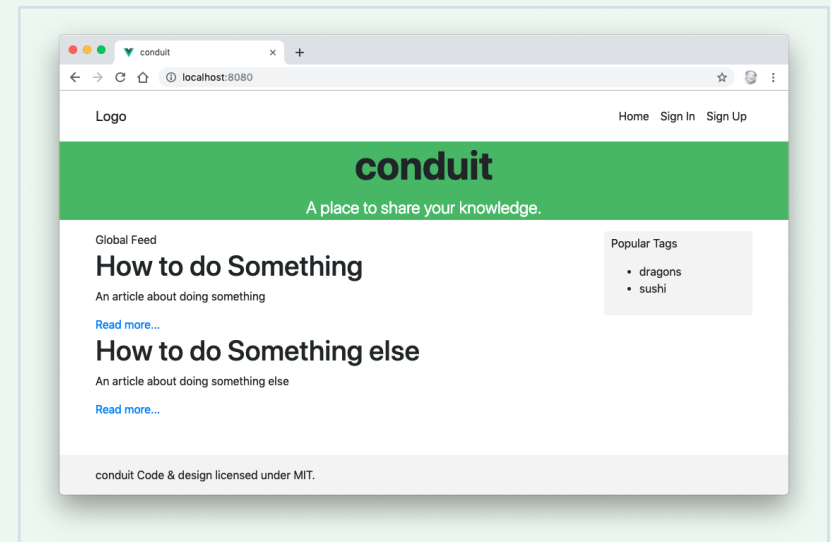
```
const fullName = computed({  
  get: () => `${firstName.value} ${lastName.value}`,  
  set: (newValue) => {  
    const names = newValue.split(' ')  
    firstName.value = names[0]  
    lastName.value = names.at(-1)  
  },  
})
```

```
fullName.value = 'Grace Hopper' // calls set()
```


Computed Properties

Use a computed property to build each article's link.

- In ArticlePreview, capture the return value of **defineProps**
- Create an **articleLink** computed property from **props.article.slug**
- Bind the "Read more..." link with **:href** (or **:to** later)
- Challenge: format the article's **createdAt** date



Reactive State

A ref is the default choice: reactive() can't hold a primitive, and loses reactivity if you destructure it.

```
import { reactive, ref } from 'vue'

const count = ref(0)           // any value; use .value
const user = reactive({ name: '' }) // objects only; no .value
```

ref and reactive

- State lives in refs, declared at the top of `<script setup>`
- When a ref changes, every component that read it re-renders
- Vue 3 tracks state with a **Proxy**, so properties added or deleted after creation are reactive too
- `Vue.set` and `vm.$set` existed only to work around that limit in Vue 2, and were removed

Vue.js Data Binding

- **v-bind** (shorthand **:**) binds an attribute or prop to a reactive value — **one-way**
 - Vue keeps the attribute up to date with the value
- **v-model** creates **two-way** binding between an input and a ref

```
  
<input v-model="searchText">
```

```
<!-- state → DOM -->  
<!-- state ↔ DOM -->
```

v-bind directive

```
<!-- bind an attribute -->  

```

```
<!-- dynamic attribute name -->  
<button v-bind:[key]="value"></button>
```

```
<!-- inline string concatenation -->  

```

```
<!-- bind several attributes from an object -->  
<input v-bind="{ id: fieldId, placeholder: hint }">
```

```
<!-- same-name shorthand (Vue 3.4+): :id="id" -->  
<div :id></div>
```

Value Binding

- `v-model` gives checkboxes, radio buttons and select options a value
- Use `v-bind` to make that value dynamic, or something other than a string

```
<!-- toggle is 'yes' or 'no' instead of true / false -->  
<input type="checkbox" v-model="toggle"  
      true-value="yes" false-value="no">
```

```
<!-- pick is set to the value of the variable a -->  
<input type="radio" v-model="pick" :value="a">
```

```
<!-- selected is set to the object, not a string -->  
<select v-model="selected">  
  <option :value="{ id: 1 }">One</option>  
</select>
```

DAY 2

2-way Binding

v-model, arrays, and the reactivity caveats that remain

v-model directive

- Creates two-way data binding on `<input>`, `<textarea>` and `<select>` elements

```
<input v-model="message">
<p>Message is: {{ message }}</p>
```

- Picks the right property and event for each element type
 - text and textarea: `value` + `input`
 - checkbox and radio: `checked` + `change`
 - select: `value` + `change`
- `v-model` ignores the initial `value`, `checked` or `selected` attributes
 - your state is the source of truth

Working with Arrays

- Every array mutation is reactive, because Vue 3 observes the array through a Proxy:
 - `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`
 - `items[0] = value` — assignment by index
 - `items.length = 0`
- Vue 2 couldn't detect the last two, and needed `Vue.set` and `splice` as workarounds
- Replacing the array wholesale is still often clearer:

```
items.value = items.value.filter((item) => !item.done)
```

Reactivity Caveats

Proxy-based reactivity is deep and covers add and delete, but a few things still break tracking:

- **Destructuring a `reactive()` object** copies the values out
 - Use `toRefs()`, or a `ref`, instead
- **Replacing a `reactive()` object wholesale** drops the old proxy
- **Reassigning a ref without `.value`** inside `<script setup>`
- **Passing `props.x` to a function** passes the current value, not a live reference

Map, Set, Date and class instances are supported, but a plain object is usually the simpler choice.

Destructuring and Reactivity

```
const state = reactive({ count: 0 })  
  
const { count } = state           // a plain 0: not reactive  
  
const { count } = toRefs(state)   // a ref: still live  
count.value++                     // updates state.count too
```

The same rule applies to Pinia stores, which is why you destructure them through `storeToRefs()`.

v-bind Shorthand

<!-- full syntax -->

`<a v-bind:href="url"> ... </a>`

<!-- shorthand -->

`<a :href="url"> ... </a>`

<!-- shorthand with a dynamic argument -->

`<a :[key]="url"> ... </a>`

<!-- same-name shorthand (3.4+) -->

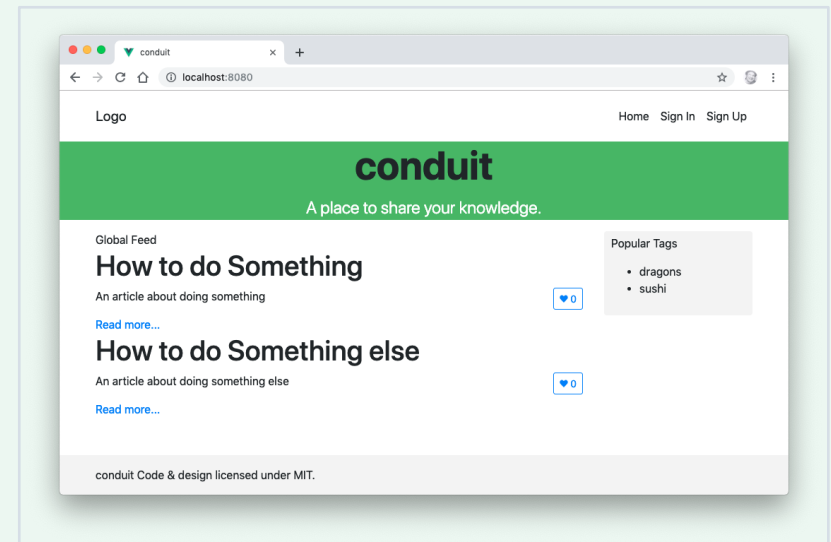
`<a :href> ... </a>`

In real Vue code you'll almost always see `:.`

Methods and State

Add a favorite button to each article, toggled by a function that updates a ref.

- Add a **favorited** ref to ArticlePreview
- Write **toggleFavorite()** — no **methods** object needed
- Add a button with **@click="toggleFavorite"**
- Switch its style with **:class** (**btn-primary** / **btn-outline-primary**)
- Challenge: add a favorites count



Event Handling

- The **v-on** directive (shorthand **@**) attaches an event listener

```
<button @click="counter += 1">Add 1</button>  
<p>The button has been clicked {{ counter }} times.</p>
```

```
import { ref } from 'vue'  
  
const counter = ref(0)
```

- Vue removes the listener for you when the element is destroyed

Inline Event Handling

Use small bits of JavaScript directly in the value of **v-on**.

```
<button @click="count++">Add 1</button>
<button @click="say('hi')">Say hi</button>

<!-- pass the native event with $event -->
<button @click="warn('Not yet', $event)">Submit</button>

<!-- or use an inline arrow function -->
<button @click="(event) => warn('Not yet', event)">Submit</button>
```

Event Handling with Methods

Declare a function in `<script setup>` and use it with `v-on`.

```
const name = ref('Vue.js')

function greet(event) {
  alert(`Hello ${name.value}!`)
  if (event) {
    alert(event.target.tagName)
  }
}
```

```
<button @click="greet">Greet</button>
```

- When you pass the function by name, Vue calls it with the native event

Event Modifiers

`v-on` modifiers handle common DOM event details for you:

- `.stop` — `event.stopPropagation()`
- `.prevent` — `event.preventDefault()`
- `.self` — only if `event.target` is the element itself
- `.capture` — listen in the capture phase (outer handlers run first)
- `.once` — run the handler at most once
- `.passive` — tell the browser you won't call `preventDefault()` (smoother scrolling)

```
<form @submit.prevent="onSubmit">...</form>
<input @keyup.enter="search">
```

v-on Shorthand

<!-- full syntax -->

```
<a v-on:click="doSomething"> ... </a>
```

<!-- shorthand -->

```
<a @click="doSomething"> ... </a>
```

<!-- shorthand with a dynamic event name -->

```
<a @[event]="doSomething"> ... </a>
```

Listening to Child Component Events

- Child components **declare** and **emit** events

```
<!-- BlogPost.vue -->
<script setup>
const emit = defineEmits(['enlarge-text'])
</script>
<template>
  <button @click="emit('enlarge-text')">Enlarge text</button>
</template>
```

- Parent components **listen** for them

```
<BlogPost @enlarge-text="postFontSize += 0.1" />
```

An undeclared listener falls through to the child's root element as a native attribute, so always declare your events.

Emitting a Value With an Event

- `emit` takes extra arguments: the event's payload

```
const emit = defineEmits(['enlarge-text'])
```

```
<button @click="emit('enlarge-text', 0.1)">Enlarge text</button>
```

- The parent receives them as handler arguments

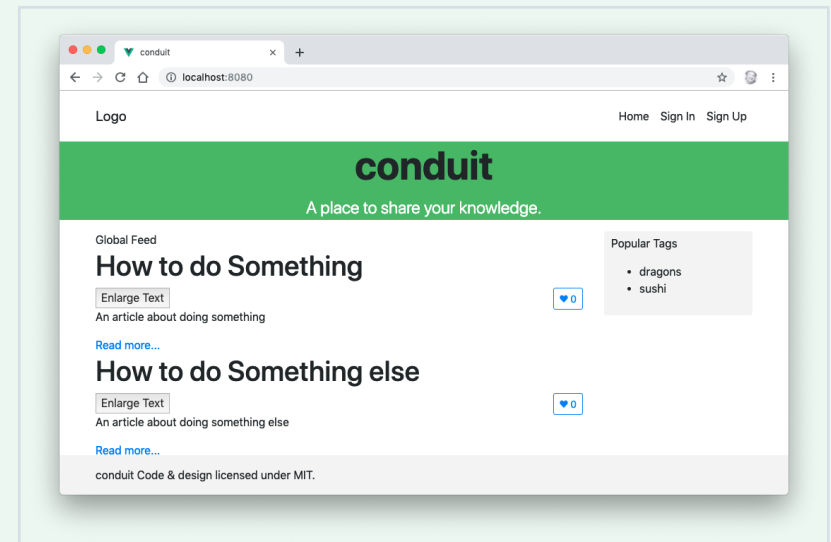
```
<BlogPost @enlarge-text="postFontSize += $event" />  
<BlogPost @enlarge-text="(n) => (postFontSize += n)" />  
<BlogPost @enlarge-text="enlarge" />    <!-- function enlarge(n) -->
```

Vue 2's `$on` / `$off` / `$once` "event bus" was removed in Vue 3. Use a store or provide/inject instead.

Events

Let each article preview ask its parent to change the font size of the whole list.

- Add an `articleFontSize` ref to `ArticleList`
- Bind it with `:style` on each `ArticlePreview`
- Declare `enlarge-text` with `defineEmits` in `ArticlePreview`
- Emit it from an "Enlarge Text" button
- Listen with `@enlarge-text` in `ArticleList`; add "Shrink Text"



Watchers

- A general way to react to state changes with **side effects**
- Most useful for asynchronous or expensive work in response to changing data

```
import { ref, watch } from 'vue'

const question = ref('')
const answer = ref('Ask me a question!')

watch(question, async (newQuestion, oldQuestion) => {
  if (!newQuestion.endsWith('?')) return
  answer.value = 'Thinking...'
  answer.value = await getAnswer(newQuestion)
})
```

watch and watchEffect

watch(source, callback)

- Explicit about what it tracks; gives you the new **and** old value
- **Lazy**: doesn't run until the source changes (unless `{ immediate: true }`)

watchEffect(callback)

- Tracks whatever the callback reads, automatically; runs once **immediately**

```
watchEffect(() => {  
  document.title = `${count.value} new posts`  
})
```

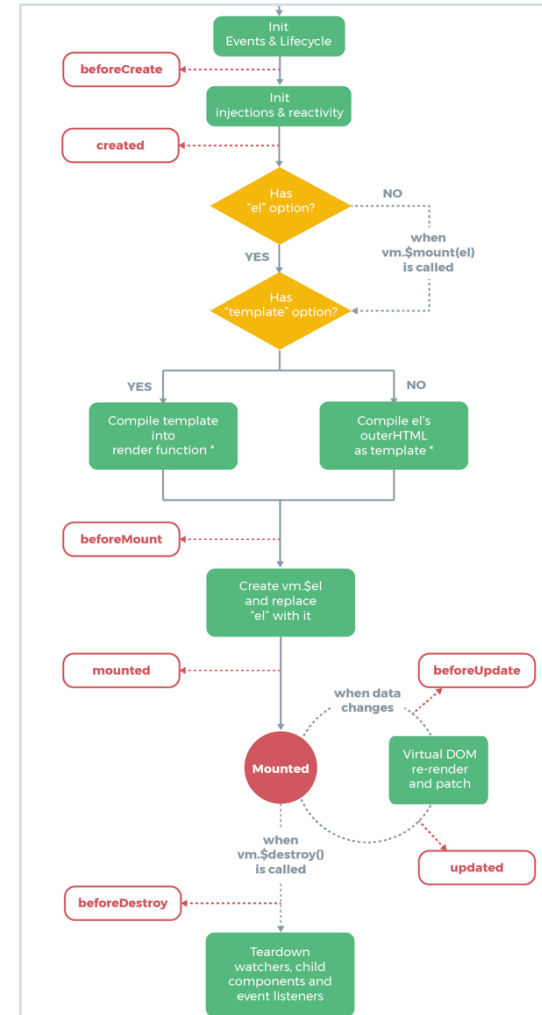
Both return a stop function and stop automatically when the component unmounts. Reach for `computed()` first — a watcher is for side effects.

Component Lifecycle

Functions imported from **vue** and called inside `<script setup>`:

- `onBeforeMount` / `onMounted`
- `onBeforeUpdate` / `onUpdated`
- `onBeforeUnmount` / `onUnmounted`
- `onErrorCaptured`, `onActivated`, `onDeactivated`

There's no `onCreated`: the body of `<script setup>` is that hook. Vue 2 named the unmount pair `beforeDestroy` and `destroyed`.



Setup (replacing beforeCreate / created)

- The body of `<script setup>` runs once per component instance, before it's mounted
- State and props exist; there's **no DOM yet**
- This is where you:
 - declare refs, computed values and functions
 - register lifecycle hooks and watchers
 - start any request whose result the first render doesn't need

```
const articles = ref([])
const loading = ref(true)
fetchArticles().then((list) => {
  articles.value = list
  loading.value = false
})
```

onBeforeMount

- Runs after setup, right before the first render is inserted into the DOM

```
import { onBeforeMount } from 'vue'

onBeforeMount(() => {
  console.log('about to render')
})
```

- Rarely needed: setup already runs before mounting
- Not called during server-side rendering

onMounted

- The component's DOM has been created and inserted
- Full access to the rendered DOM through **template refs**
- Use for measuring elements, focusing inputs, and integrating non-Vue libraries

```
const input = ref(null)           // <input ref="input">

onMounted(async () => {
  input.value.focus()
  articles.value = await fetchArticles()
})
```

- Children are mounted before their parent

onBeforeUpdate

- Runs after state changes, **before** the DOM is patched
- Use it to read the DOM as it was before the update
 - for example, to record a scroll position

```
let previousHeight = 0

onBeforeUpdate(() => {
  previousHeight = list.value.scrollHeight
})
```

onUpdated

- Runs after a state change has re-rendered and patched the DOM
- Don't change state here without a guard — you'll create an update loop
- To react to one specific value, prefer `watch()`

```
onUpdated(() => {  
  const el = list.value  
  if (wasAtBottom) el.scrollTop = el.scrollHeight  
})
```

onBeforeUnmount

- Fires right before teardown, while the component is still fully functional
- Clean up anything **you** created that Vue doesn't know about:
 - timers (`setInterval`)
 - subscriptions (WebSocket, event source)
 - listeners on `window` or `document`
 - instances of non-Vue libraries (charts, maps, editors)

onUnmounted

- Everything attached to the component has been torn down and removed from the DOM
- Watchers and lifecycle hooks are stopped for you
- Anything you set up by hand is **not**
- `onBeforeUnmount` and `onUnmounted` are both fine for cleanup; use `onBeforeUnmount` if you need the DOM

Lifecycle Hooks in Practice

```
<script setup>
import { onMounted, onUnmounted, ref } from 'vue'

const seconds = ref(0)
let timer

onMounted(() => {
  timer = setInterval(() => seconds.value++, 1000)
})

onUnmounted(() => clearInterval(timer))
</script>

<template>
  <p>You've been here {{ seconds }} seconds.</p>
</template>
```


Using Lifecycle Hooks

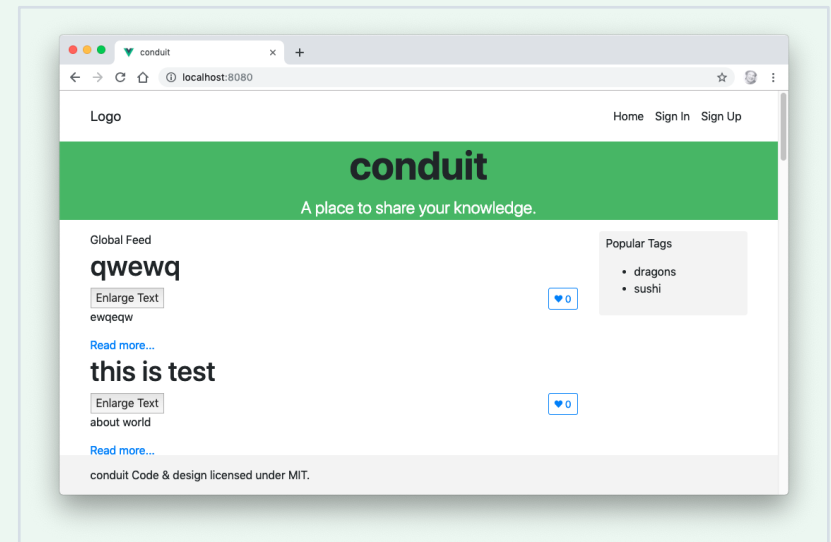
- Hooks must be called **synchronously** in the body of `<script setup>`
 - Not inside a callback, a promise handler, or a condition
 - That's how Vue knows which component instance to attach them to
- They can be called from a **composable**, because a composable is itself called synchronously from setup

```
// X too late: Vue no longer knows the current component  
setTimeout(() => onMounted(() => {}), 0)
```

Component Lifecycle

Load articles from the real Conduit API when ArticleList mounts.

- Create `src/common/config.js` with `API_URL` (from `import.meta.env.VITE_API_URL`)
- `npm install axios`
- Create `api.service.js` with an axios instance and a `query()` method
- Fetch articles in `onMounted` and assign them to the `articles` ref
- Remove the JSON import; start with `ref([])`



DAY 2

Vue and Forms

Text inputs, checkboxes, radio buttons, selects and modifiers

Input Bindings

```
<input v-model="yourAnswer">
<textarea v-model="yourAnswer"></textarea>

<input type="checkbox" id="agree" v-model="checked">

<input type="radio" id="first" value="First" v-model="picked">

<select v-model="selected">
  <option disabled value="">Please select one</option>
  <option>A</option>
  <option>B</option>
  <option>C</option>
</select>
```

2-way Binding Pros and Cons

- 2-way binding makes development easier
- 2-way binding can make data flow harder to follow and test
- `v-model` is a little bit magic (which could be a pro or a con)

```
<input v-model="searchText">
```

- ...does the same thing as:

```
<input  
  :value="searchText"  
  @input="searchText = $event.target.value">
```

Creating a Form with Checkboxes

Files: `demos/forms/checkboxes.html`

- A single checkbox bound to a **boolean**
- Several checkboxes bound to one **array**
- **true-value** / **false-value** for custom values

```
<input type="checkbox" value="Vue" v-model="frameworks">  
<input type="checkbox" value="React" v-model="frameworks">
```

Creating a Form with Radio Buttons

Files: demos/forms/radio.html

- Radio buttons in a group share one `v-model`
- The ref holds the `value` of the selected button
- Use `:value` to select numbers or objects

```
<input type="radio" value="small" v-model="size">  
<input type="radio" value="large" v-model="size">
```

Creating a Form with a Select Element

Files: demos/forms/select.html

- Single select: the ref holds the chosen option's value
- **multiple**: the ref holds an array
- Generate options with **v-for**; bind objects with **:value**

```
<select v-model="country">
  <option disabled value="">Choose...</option>
  <option v-for="c in countries" :key="c.code" :value="c.code">
    {{ c.name }}
  </option>
</select>
```


Input Modifiers

Files: demos/forms/modifiers.html

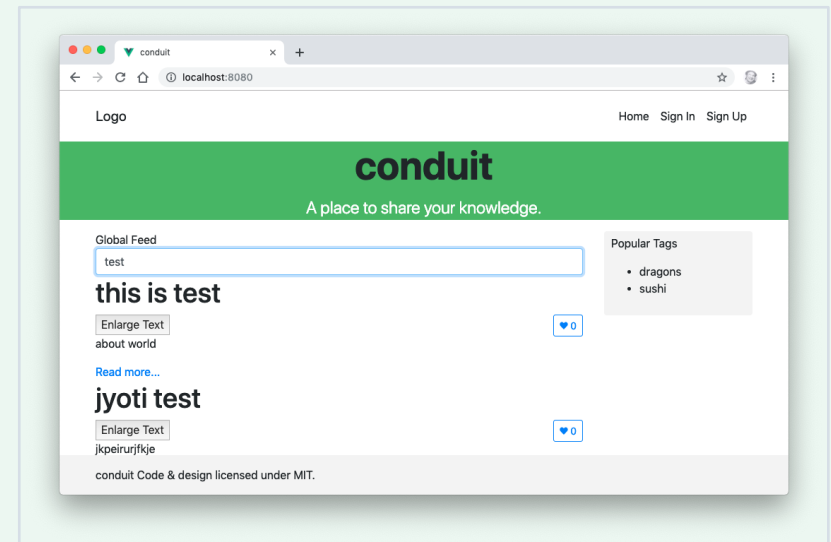
- `.lazy` — sync on `change` instead of every `input`
- `.number` — cast the value to a number
- `.trim` — strip leading and trailing whitespace

```
<input v-model.lazy="message">  
<input v-model.number="age">  
<input v-model.trim="username">
```

Forms

Filter the article list as the user types.

- Add an input bound to a **searchDetails** ref with **v-model**
- Create a **filterIt** computed property
- Loop over **filterIt** instead of **articles**
- Challenges: make it case-insensitive; search the description too



DAY 3

Vue Components

Props, naming, registration, v-model, slots and async components

What are Vue Components?

- A reusable piece of UI with its own state, template and styles
- Written as a single-file component: a `.vue` file with `<script setup>`, `<template>` and `<style>`
- State is declared with `ref()` and `reactive()`, so each instance gets its own copy
 - That's what the `data()` function guaranteed in the Options API
- A component may have more than one root element (a **fragment**)
- Components form a tree: data flows **down** as props, events flow **up**

Creating a Component

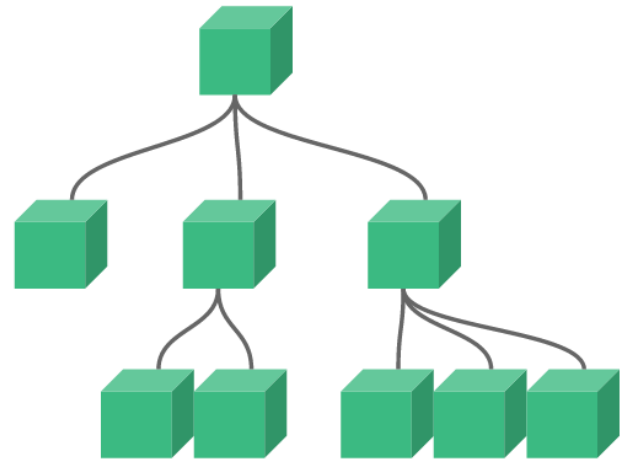
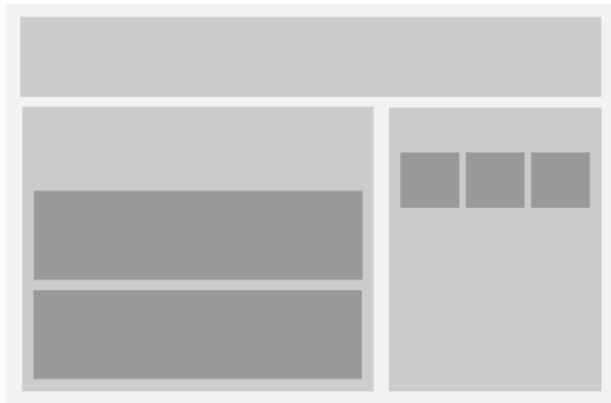
```
<!-- TodoItem.vue -->
<script setup>
defineProps({
  todo: { type: Object, required: true },
})
</script>

<template>
  <li>{{ todo.text }}</li>
</template>
```

- The file name is the component name: `TodoItem.vue` is used as `<TodoItem />`
- Vue 2 wrote `Vue.component('todo-item', { ... })`

Organizing and Reusing Components

Apps are usually organized as a tree of nested components.



Naming Components

- **PascalCase** — the recommended default
 - Matches the file name, and the editor can resolve it
 - Distinguishes your components from native HTML elements
- **kebab-case** — `<todo-item>`
 - Required when templates are written directly in the HTML page (in-DOM templates), because HTML tag names are case-insensitive
- Use **multi-word** names (`TodoItem`, not `Todo`) to avoid clashing with current and future HTML elements

Local Components

The usual case: import the component where you use it.

```
<script setup>
import PopupWindow from './components/PopupWindow.vue'
</script>

<template>
  <PopupWindow />
</template>
```

- The import **is** the registration. There's no **components** option.

Global Components

```
// main.js  
import { createApp } from 'vue'  
import App from './App.vue'  
import PopupWindow from './components/PopupWindow.vue'  
  
const app = createApp(App)  
  
app.component('PopupWindow', PopupWindow)  
  
app.mount('#app')
```

- Registered on the app, not on a global **Vue**
- Usable in every component's template without an import

Global vs. Local

Prefer local registration.

- A reader can see where every component comes from
- Unused components can be tree-shaken out of the bundle
- The editor can jump to the definition and type-check props

Register globally only for components used on nearly every screen — an icon, or a base button.

Props

Declare props with the `defineProps` compiler macro.

```
const props = defineProps({
  todo: { type: Object, required: true },
  size: { type: String, default: 'medium' },
})
```

- In the template, write `todo` directly; in script, use `props.todo`
- Props are **read-only** — emit an event to ask the parent for a change
- `type`, `required`, `default` and `validator` produce dev-time warnings

Passing Data

- Pass data from parent to child components using **props**
- Pass props using attributes:

```
<!-- a static string -->  
<TodoItem todo="wash the car" />  
  
<!-- any JavaScript value: use v-bind -->  
<TodoItem :todo="item" :priority="2" :done="false" />
```

- A component can have as many props as you need
- Any value can be passed to any prop

Using Props

Parent

```
<TodoItem :todo="item" size="large" />
```

Child: TodoItem.vue

```
<script setup>
defineProps(['todo', 'size'])
</script>

<template>
  <li :class="size">{{ todo.text }}</li>
</template>
```

Fragments: Multiple Root Elements

Vue 2 required a single root element. Vue 3 doesn't.

```
<template>
  <h1>{{ title }}</h1>
  <div v-html="content"></div>
</template>
```

- With more than one root, Vue can't decide where to put inherited attributes (**class**, **style**, listeners)
- Bind **\$attrs** explicitly if the parent passes them:

```
<div v-bind="$attrs" v-html="content"></div>
```

Using v-model on Components

`v-model` works on components as it does on inputs:

```
<CustomInput v-model="searchText" />
```

...does the same thing as:

```
<CustomInput  
  :model-value="searchText"  
  @update:model-value="(v) => (searchText = v)" />
```

- Vue 3 uses the `modelValue` prop and `update:modelValue` event (Vue 2 used `value` and `input`)
- Named models allow several per component: `v-model:title`, `v-model:body`

defineModel

Since Vue 3.4, the whole prop-and-event pair has a shorthand.

```
<!-- CustomInput.vue -->
<script setup>
const model = defineModel()
</script>

<template>
  <input v-model="model">
</template>
```

- `model` is a writable ref
- Writing to it emits `update:modelValue` for you
- Named and typed: `defineModel('title', { type: String, required: true })`

Content Distribution with Slots

Slots let a parent pass **template content** into a component.

```
<!-- parent -->
<NavLink url="/profile">
  Your Profile
</NavLink>
```

```
<!-- NavLink.vue -->
<script setup>
defineProps({ url: String })
</script>

<template>
  <a :href="url" class="nav-link">
    <slot></slot>
  </a>
</template>
```

Cool Things You Can Do With Slots

- Slots can contain any template code, including HTML and components
- Slot content can use the **parent's** state (it's compiled in the parent's scope)
- Provide **default content** for when the parent doesn't supply any:

```
<button class="btn">  
  <slot>Submit</slot>  
</button>
```

- Use **named slots** when a component needs several insertion points

Naming Slots

The parent fills each named slot with a `<template v-slot:name>`:

```
<BaseLayout>
  <template v-slot:header>
    <h1>Header content</h1>
  </template>

  <p>Main content goes in the default slot.</p>

  <template v-slot:footer>
    <p>Footer content</p>
  </template>
</BaseLayout>
```

Using Named Slots

```
<!-- BaseLayout.vue -->
<template>
  <div class="container">
    <header>
      <slot name="header"></slot>
    </header>
    <main>
      <slot></slot>           <!-- the default slot -->
    </main>
    <footer>
      <slot name="footer"></slot>
    </footer>
  </div>
</template>
```

Using Slots

Files: `demos/slots/index.html`

- A **BaseCard** component with **header**, **default** and **footer** slots
- Default content when a slot is empty
- A scoped slot: **ItemList** passes each **item** back to the parent
- Open the file in a browser; edit the template to experiment

More Fun with Slots

- **Named slot shorthand:** `v-slot:header` can be written `#header`

```
<template #header>
  <p>template content here</p>
</template>
```

- **Scoped slots:** the child passes data to the slot content

```
<!-- ItemList.vue -->
<li v-for="item in items" :key="item.id">
  <slot name="item" :item="item"></slot>
</li>

<!-- parent -->
<ItemList :items="todos">
  <template #item="{ item }">{{ item.text }}</template>
</ItemList>
```

- Also: dynamic slot names (`#[name]`), abbreviated default slot syntax

Slots

Load popular tags from the API and display them, and each article's tags, with slots.

- Create **TagItem** and **TagList** (with a default slot and fallback text)
- Load tags from `/tags` (same base URL as articles)
- Add a named `tags` slot to `ArticlePreview`; fill it with `#tags`
- Make clicking a tag filter the article list

Loading Components Asynchronously

Wrap a dynamic import with `defineAsyncComponent`:

```
import { defineAsyncComponent } from 'vue'

const AsyncModal = defineAsyncComponent(() =>
  import('./components/Modal.vue')
)
```

- Vite gives the component its own chunk, downloaded on first use
- Routes get this for free from a lazy route component
- Options for a loading component, an error component, delay and timeout
- `<Suspense>` can show a fallback while async components load (still marked experimental)

DAY 3

Composables

Reusing stateful logic with plain functions

Composable Basics

- A **composable** is a function that uses the Composition API to encapsulate and reuse **stateful** logic
- By convention it's named **use...** and lives in **src/composables**
- It's called synchronously from **<script setup>**, so it can use lifecycle hooks and watchers on the caller's behalf

```
const { x, y } = useMouse()    // reactive mouse position
```

Why Composables Replace Mixins

A mixin merged its options into the component silently:

- You couldn't tell where an inherited property came from
- Two mixins declaring the same name collided, and a merge strategy picked the winner
- A mixin couldn't take arguments

A composable takes its inputs as **parameters** and returns its outputs, so both ends are visible at the call site.

Mixins still work in Vue 3, but the docs describe them as legacy.

Writing a Composable

```
// src/composables/useArticleFilter.js
import { computed, ref } from 'vue'

export function useArticleFilter(articles) {
  const searchDetails = ref('')

  const filterIt = computed(() =>
    articles.value.filter((a) =>
      a.title.includes(searchDetails.value)
    )
  )

  return { searchDetails, filterIt }
}
```

Using a Composable

```
<script setup>
import { ref } from 'vue'
import { useArticleFilter } from '@/composables/useArticleFilter'

const articles = ref([])

const { searchDetails, filterIt } = useArticleFilter(articles)
</script>

<template>
  <input v-model="searchDetails">
  <ArticlePreview v-for="a in filterIt" :key="a.slug" :article="a" />
</template>
```

Called twice, you get two independent filters — something a mixin could never do.

Composable Conventions

- **Name it `use...`** so readers know it may register hooks
- **Call it synchronously from `setup`**, never in a callback or after `await`
- **Accept refs or getters** as arguments so callers can pass live state; `toValue()` normalizes them
- **Return plain refs**, so the caller can destructure freely
- **Clean up after yourself** in `onUnmounted`

Community Composables

- **VueUse** — a large collection of ready-made composables
 - `useLocalStorage`, `useFetch`, `useMediaQuery`, `useDebounceFn`, `useIntersectionObserver`, ...
- Worth checking before writing your own
- **vueuse.org**

```
import { useLocalStorage } from '@vueuse/core'

const theme = useLocalStorage('theme', 'light') // a ref, persisted
```

Composables

Extract the article filter into a reusable composable.

- Create `src/composables/useArticleFilter.js`
- Move `searchDetails` and `filterIt` into it
- Call it from `ArticleList`: `const { searchDetails, filterIt } = useArticleFilter(articles)`
- Confirm the filter still works
- Challenge: reuse it to filter the tags

DAY 3

Using Pinia

Vue's official state management library

DAY 3

Why a Store?

The Problem

- Props flow down; events flow up
- That works until two distant components need the same state:
 - Passing it through every intermediate component ("prop drilling")
 - Or keeping two copies that drift out of sync
- A **store** gives that state one owner, outside the component tree
- Any component can read it, and changes go through clearly named **actions**

Pinia

- Vue's official state management library
- A store is a composable: you call `useSomeStore()` to get it
- **state**, **getters** and **actions** — no mutations, no `dispatch`, no `commit`
- Type-safe, and each store is its own module by construction
- Replaced **Vuex**, which is now in maintenance mode

Pinia vs. Vuex

Vuex	Pinia
state	state
getters	getters
mutations	— (actions assign directly)
actions + <code>dispatch('name')</code>	actions, called as functions
<code>commit('NAME', payload)</code>	<code>this.value = payload</code>
modules + namespacing	one file per store
<code>mapState</code> / <code>mapGetters</code>	<code>storeToRefs</code>

Installing Pinia

```
npm install pinia
```

```
// main.js  
import { createApp } from 'vue'  
import { createPinia } from 'pinia'  
import App from './App.vue'  
  
const app = createApp(App)  
app.use(createPinia())  
app.mount('#app')
```

Install Pinia before the router, so navigation guards can reach the stores.

Defining a Store

```
// src/stores/counter.js
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', {
  state: () => ({ count: 0 }),

  getters: {
    double: (state) => state.count * 2,
  },

  actions: {
    increment() {
      this.count++
    },
  },
})
```

- **'counter'** is the store's unique **id**, shown in the DevTools

Setup Stores

A store can also be written like `<script setup>`:

```
export const useCounterStore = defineStore('counter', () => {  
  const count = ref(0) // state  
  const double = computed(() => count.value * 2) // getter  
  
  function increment() { // action  
    count.value++  
  }  
  
  return { count, double, increment }  
})
```

- Same store, different syntax
- Setup stores compose more easily; option stores map directly onto a Vuex module you're porting

State

- `state` is a function that returns a fresh object
- Read and write it directly on the store:

```
const counter = useCounterStore()

counter.count++
counter.$patch({ count: 10 })    // several changes at once
counter.$reset()                 // back to the initial state
```

- `$reset()` is built in for option stores; setup stores define their own

Getters

- Computed properties for a store
- Receive state as their first argument
- Cached, like any computed property

```
getters: {  
  doneTodos: (state) => state.todos.filter((t) => t.done),  
  doneCount() {  
    return this.doneTodos.length  
  },  
},
```

- Use `this` (and a regular function) to reach other getters

Using a Store in a Component

```
<script setup>
import { useCounterStore } from '@stores/counter'

const counter = useCounterStore()
</script>

<template>
  <p>{{ counter.count }} / {{ counter.double }}</p>
  <button @click="counter.increment()">+1</button>
</template>
```

storeToRefs

A store is reactive, so destructuring it copies the values out and breaks reactivity.

```
import { storeToRefs } from 'pinia'

const { count, double } = counter           // ✗ not reactive

const { count, double } = storeToRefs(counter) // ✓ refs
const { increment } = counter               // ✓ actions
```

storeToRefs replaces **mapState** and **mapGetters**.

Actions

- An action is just a method on the store
- It can be **async**, and it changes state through **this**

```
actions: {  
  async fetchArticles(params) {  
    this.isLoading = true  
    try {  
      const { data } = await ArticlesService.query(params.type)  
      this.articles = data.articles  
    } finally {  
      this.isLoading = false  
    }  
  },  
},
```

No Mutations

- Vuex separated synchronous **mutations** from asynchronous **actions** so the devtools could record every state change
- Pinia tracks changes through the reactivity system instead, so that separation is unnecessary
- The result: no mutation types, no **commit**, no string constants — one less indirection between the component and the state
- **\$subscribe** and **\$onAction** are there if you need to observe changes (logging, persistence)

Multiple Stores

Every store is a separate module by construction.

```
src/stores/  
  auth.js    →  useAuthStore  
  home.js    →  useHomeStore
```

- A store can use another store — just call its **use...** function inside an action or getter

```
actions: {  
  async fetchFeed() {  
    const auth = useAuthStore()  
    if (!auth.isAuthenticated) return  
    // ...  
  },  
},
```

Vuex needed modules, namespacing, and **'module/action'** strings.

Pinia and the DevTools

- Vue DevTools shows every registered store
 - Inspect and edit state live
 - A timeline of actions, with arguments and results
 - Time-travel: jump back to an earlier state
- Stores appear as soon as they're first used

Persisting Store State

Pinia is extended with **plugins**.

```
// main.js  
const pinia = createPinia()  
  
pinia.use(({ store }) => {  
  store.$subscribe((_mutation, state) => {  
    localStorage.setItem(store.$id, JSON.stringify(state))  
  })  
})
```

- A plugin runs for every store when it's created
- **pinia-plugin-persistedstate** packages this up, with options for which stores and paths to persist

Testing a Store

A store is a plain function — no component needed.

```
import { beforeEach, expect, it } from 'vitest'
import { createPinia, setActivePinia } from 'pinia'
import { useCounterStore } from '@/stores/counter'

beforeEach(() => setActivePinia(createPinia()))

it('increments', () => {
  const counter = useCounterStore()
  counter.increment()
  expect(counter.count).toBe(1)
})
```

`@pinia/testing` adds `createTestingPinia()`, which stubs actions for component tests.

Implementing Pinia

Move the article list into a Pinia store and load it with an action.

- `npm install pinia`; install it in `main.js`
- Add `ArticlesService` to `api.service.js`
- Create `stores/home.js`: state, plus `fetchArticles`, `setTags` and `updateArticleInList` actions
- In `ArticleList`: `useHomeStore()` + `storeToRefs()`; call `fetchArticles` in `onMounted`
- Watch the store in the DevTools Pinia tab

DAY 3

Single-Page Applications

Routing with Vue Router

Creating an SPA with Vue Router

- Map **URLs** to **components**, and tell Vue Router where to render them
- Clicking a link changes the URL with the History API — no page reload
- The back and forward buttons, bookmarks and deep links all work
- Route components can be **lazy-loaded**, so each page gets its own chunk
- Vue Router 4+ is the version for Vue 3

Vue Router: the Template

```
<script setup>
import { RouterLink, RouterView } from 'vue-router'
</script>

<template>
  <nav>
    <RouterLink :to="{ name: 'home' }">Home</RouterLink>
    <RouterLink :to="{ name: 'login' }">Sign in</RouterLink>
  </nav>

  <RouterView />
</template>
```

- **RouterLink** renders an `<a>` and adds an active class when its route matches
- **RouterView** renders the component for the current route

Vue Router: the JavaScript

```
// src/router/index.js
import { createRouter, createWebHistory } from 'vue-router'

export default createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { name: 'home', path: '/',
      component: () => import('@/components/HomeContent.vue') },
    { name: 'login', path: '/login',
      component: () => import('@/components/LoginPage.vue') },
  ],
})
```

- Install it in `main.js` with `app.use(router)`
- Vue Router 3 wrote `new Router({ mode: 'history', routes })`

Routing

Add Vue Router with home and sign-in routes.

- `npm install vue-router`; create `src/router/index.js`
- Add a lazy-loaded `home` route; install the router after Pinia
- Replace `<HomeContent />` in `App.vue` with `<RouterView />`
- Turn the header links into `<RouterLink>`s
- Add a `login` route and a `LoginPage` component
- Challenge: the Sign Up page

AJAX with Pinia

Add sign up, log in and authentication state with an auth store.

- Copy `jwt.service.js`, `api.service.js` and `stores/auth.js` from `labs/lab22`
- Read through the auth store: state, `login`, `register`, `checkAuth`
- Add `LoginPage` and `SignUpPage`; navigate with `useRouter()`
- Use `isAuthenticated` (via `storeToRefs`) in `HomeContent`
- Challenges: a logout button; `checkAuth()` on startup

DAY 3

Testing

TDD, Vitest and Vue Test Utils

Test-Driven Development

Objectives

- Learn the TDD steps
- Write assertions
- Create tests with Vitest
- Test components with Vue Test Utils

Goal of TDD

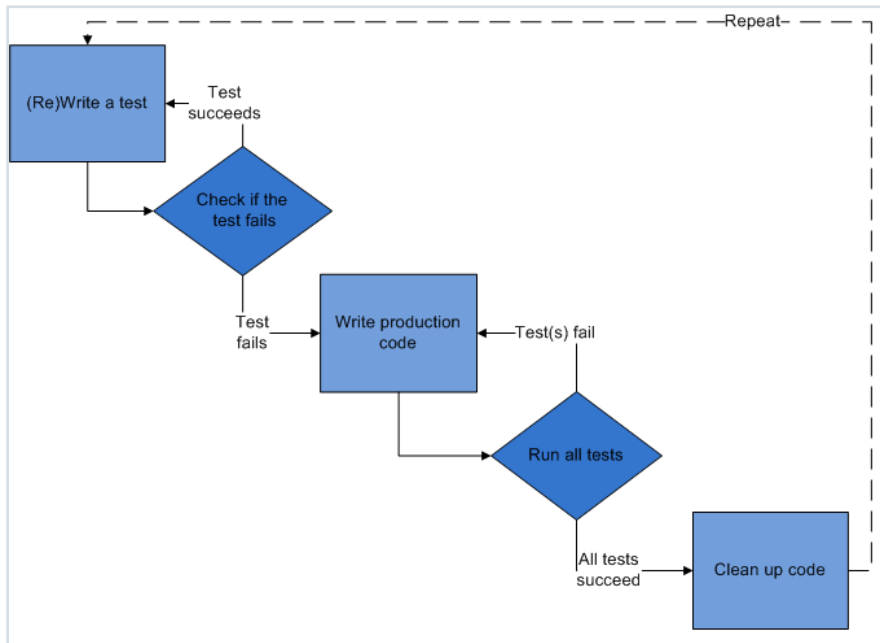
Clean code that works. — Kent Beck

- *Works*: every behavior is pinned down by a test
- *Clean*: you can refactor fearlessly, because the tests tell you if you broke something

The TDD Cycle

1. **Red** — write a little test that doesn't work
2. **Green** — make the test pass as quickly as possible
 - Don't worry about doing it right
3. **Refactor** — eliminate the duplication created in making the test pass
 - Tests stay green the whole time

TDD Steps



1. Write a test
2. Check that the test fails
3. Write code
4. Run tests — pass!
5. Refactor
6. Repeat

Red

- Write the story
- Invent the interface you wish you had
- Characteristics of good tests:
 - Each test is independent of the others
 - Any behavior is specified in only one test
 - No unnecessary assertions
 - Test only one unit of code at a time
 - Avoid unnecessary preconditions

Green

Get the test to pass as quickly as possible. Three strategies:

- **Fake it** — do something, no matter how crude, to make the test pass (even return a constant)
- **Obvious implementation** — if the clean solution is obvious, type it in
 - ...but don't try too hard
- **Triangulation** — only generalize when you have two or more examples
 - When the second example demands a more general solution, then and only then generalize

Refactor

- Make it right
- Remove duplication
- Improve the test
- Repeat
- Add ideas that aren't immediately needed to a **to-do list**, not the code

Assertions

- An assertion is an expression that checks a value and fails the test if it's wrong
- Assertion styles:

```
expect(buttonText).toBe( 'Go! ' )           // Vitest / Jest
expect(body).to.be.an( 'array' )            // Chai
assert.deepEqual(obj1, obj2)                 // node:assert
```

- Vitest's **expect** includes Chai's, with a Jest-compatible API on top

JavaScript Testing Frameworks

- **Vitest** — Vite-native; shares your Vite config and transforms
- **Jest** — long the default; needs its own transform setup for Vite projects
- **Mocha + Chai** — runner and assertion library, chosen separately
- **node:test** — built into Node.js since version 18
- **Playwright / Cypress** — end-to-end tests in a real browser

JS Exception Handling

```
function hello(name) {  
  return 'Hello, ' + name  
}  
  
const result = hello('World')  
const expected = 'Hello, World!'  
  
try {  
  if (result !== expected) {  
    throw new Error(`Expected ${expected} but got ${result}`)  
  }  
} catch (err) {  
  console.log(err.message)  
}
```

Vitest Overview

Objectives

- Learn about Vitest
- Write unit tests with Vitest
- Understand the rendering modes: `mount` vs. `shallowMount`
- Mock modules, functions and timers

Vitest

- The test runner built for Vite
- Reuses your `vite.config.js` — aliases, plugins and `.vue` handling all work in tests with no extra configuration
- Runs files named `*.spec.js` or `*.test.js`
- Simulates a browser environment with `jsdom` or `happy-dom`
- Jest-compatible API: `describe`, `it`, `expect`, `vi.fn()`
- Watch mode re-runs only the tests affected by your last save

How Vitest Works

- **Suites** (`describe`) group your tests
- **Specs** (`it` / `test`) contain expectations

```
import { describe, expect, it } from 'vitest'

describe('A suite is just a function', () => {
  it('and so is a spec', () => {
    expect(true).toBe(true)
  })
})
```

- Set `globals: true` in the test config to skip the import

Test Suites

- Created with the `describe` function
- Contain one or more specs (and can nest other suites)
- Two parameters:
 - a text description
 - a function

```
describe('hello', () => {  
  // specs go here  
})
```

- Setup and teardown: `beforeEach`, `afterEach`, `beforeAll`, `afterAll`

Specs

- Created with the **it** or **test** function — they're the same thing
- Contain one or more expectations (assertions)

```
describe('hello', () => {  
  it('concatenates Hello and a name', () => {  
    expect(hello('World')).toBe('Hello, World!')  
  })  
})
```

- Name specs so that a failure reads like a bug report

Expectations

- Also called assertions
- Made with the **expect** function:
 - takes the **actual** value
 - chained to a **matcher**
 - which takes the **expected** value

```
expect(actual).toEqual(expected)
```

Matchers

```
expect(fn).toThrow(error)
expect(value).toBe(other)           // Object.is (identity)
expect(value).toEqual(other)       // deep equality
expect(value).toBeDefined()
expect(value).toBeTruthy()         // or .toBeFalsy()
expect(number).toBeGreaterThan(n)  // or .toBeLessThan(n)
expect(value).toBeNull()           // or .toBeUndefined()
expect(array).toContain(member)
expect(string).toMatch(/pattern/)
await expect(promise).resolves.toBe(value)
```

- Add `.not` to negate any matcher: `expect(x).not.toBe(0)`

TDD vs. BDD

Test-Driven Development

- Focused on being useful for programmers
- Tests describe **results**
- Terse names

Behavior-Driven Development

- Focused on documentation, readable by non-programmers
- Tests describe **features and behavior**
- More verbose names ("should ...")

TDD Example

```
describe('Counter', () => {  
  test('tick increases count to 1', () => {  
    const counter = new Counter()  
  
    counter.tick()  
  
    expect(counter.count).toBe(1)  
  })  
})
```

BDD Example

```
describe('Counter', () => {  
  it('should increase count by 1 after calling tick', () => {  
    const counter = new Counter()  
    const expected = counter.count + 1  
  
    counter.tick()  
  
    expect(counter.count).toBe(expected)  
  })  
})
```

Mocking

Replace real dependencies with controllable fakes:

- `vi.fn()` — a mock function that records how it was called
- `vi.mock()` — replace a whole module
- `vi.spyOn()` — wrap an existing method, keeping the original
- `vi.useFakeTimers()` — control `setTimeout` and `setInterval`
- `vi.stubGlobal()` — replace a global such as `fetch`

Mock Function

```
import { expect, it, vi } from 'vitest'

it('records calls', () => {
  const mockCallback = vi.fn()

  ;[0, 1].forEach(mockCallback)

  // The mock function was called twice
  expect(mockCallback).toHaveBeenCalledTimes(2)

  // First call: (value 0, index 0, the array)
  expect(mockCallback).toHaveBeenCalledWith(1, 0, 0, [0, 1])
})
```


Mocking a Module

Replace a module your component imports, so the test never hits the network:

```
vi.mock('@common/api.service', () => ({  
  default: {  
    query: vi.fn().mockResolvedValue({ data: { tags: [] } }),  
  },  
}))
```

- **vi.mock** is **hoisted** above the imports, so it applies even though it's written after them

Resetting Mocks

Mocks remember every call, so state leaks between tests.

```
vi.clearAllMocks()    // forget recorded calls  
vi.resetAllMocks()    // also drop the implementations  
vi.restoreAllMocks()  // undo every spyOn
```

- Or set `clearMocks`, `mockReset` or `restoreMocks`: `true` in the Vitest config

Fake Timers

```
vi.useFakeTimers()

const spy = vi.fn()
setTimeout(spy, 5000)

vi.advanceTimersByTime(5000)    // no real waiting

expect(spy).toHaveBeenCalled()

vi.useRealTimers()
```

Snapshot Testing

1. Renders a component (or any value)
2. Writes a snapshot file on the first run
3. Compares later runs against it, and fails if the output differs
4. Update deliberately with `vitest -u`, and review the diff

Best for output that should rarely change. A poor substitute for an assertion about behavior.

Sample Snapshot Test

```
import { mount } from '@vue/test-utils'

it('renders content correctly', () => {
  const wrapper = mount(TheHeader, {
    global: { plugins: [pinia, router] },
  })

  expect(wrapper.html()).toMatchSnapshot()
})
```

- The first run creates `__snapshots__/_TheHeader.spec.js.snap`

Vue Test Utils

- The official unit-testing utility library for Vue
- **Version 2** is the one built for Vue 3
- Changes from version 1:
 - `propsData` is now `props`
 - `createLocalVue` is gone — use `global.plugins`
 - `stubs`, `mocks` and `provide` moved under `global`
 - `setValue()` and `trigger()` return a promise; `await` them

mount

Creates a wrapper containing the mounted and rendered component, **including all of its children.**

```
const wrapper = mount(TheHeader, {  
  props: { title: 'Conduit' },  
  global: { plugins: [router] },  
})  
  
expect(wrapper.text()).toContain('Conduit')
```

shallowMount

Mounts the component with its **child components stubbed out**.

- Useful when a child would fetch data or need its own plugins

```
const wrapper = shallowMount(HomeContent, {  
  global: { plugins: [pinia, router] },  
})
```

- Children render as `<global-feed-stub>` placeholders

Testing with Vitest

Fix the HomeComponent tests now that it depends on the router, Pinia and the API.

- Run `npm run test` and read the failure
- Create a test router with `createRouter` and `createWebHistory`
- Create a fresh Pinia in `beforeEach`
- `shallowMount` with `global: { plugins: [pinia, router] }`
- `vi.mock('@common/api.service')` so `/tags` isn't really fetched
- Challenge: test the auth store directly

DAY 3

Testing in a Real Browser

Browser and End-to-End Testing

- jsdom is a fast approximation of a browser, not a browser
- **Vitest browser mode** runs the same tests in a real browser, driven by Playwright or WebdriverIO
- **Playwright** and **Cypress** drive the whole application end to end
- create-vue can set up end-to-end testing for you

Karma, which earlier versions of this course covered, was deprecated in 2023.

DAY 3

Transitions and Animations

<Transition>, <TransitionGroup>, CSS classes and JavaScript hooks

<Transition>

- A built-in wrapper component
- Creates enter and leave transitions for the element or component inside it
- Triggered by:
 - conditional rendering (**v-if**)
 - conditional display (**v-show**)
 - dynamic components (**<component :is>**)
 - changing the **key** of the child
- **<TransitionGroup>** does the same for a list rendered with **v-for**

How <Transition> Works

When the wrapped element is inserted or removed, Vue:

1. Checks whether it has CSS transitions or animations; if so, adds and removes the transition classes at the right times
2. Calls any JavaScript hooks you've provided
3. If neither, inserts or removes it on the next frame

```
<button @click="show = !show">Toggle</button>

<Transition name="fade">
  <p v-if="show">hello</p>
</Transition>
```

The Transition Classes

Vue applies these classes, prefixed with the transition's **name**, in order:

- **v-enter-from** — the starting state, applied before insertion
- **v-enter-active** — applied for the whole entering phase (put **transition** here)
- **v-enter-to** — the end state of the enter transition
- **v-leave-from** — the starting state of the leave transition
- **v-leave-active** — applied for the whole leaving phase
- **v-leave-to** — the end state, applied until the animation finishes

Vue 2 called the first and fourth **v-enter** and **v-leave**. That rename is the most common reason an old example stops animating.

Defining the CSS

```
.fade-enter-active,  
.fade-leave-active {  
  transition: all 0.3s ease;  
}  
  
.fade-enter-from,  
.fade-leave-to {  
  transform: translateX(10px);  
  opacity: 0;  
}
```


JavaScript Hooks

```
<Transition
  @before-enter="beforeEnter"
  @enter="enter"
  @after-enter="afterEnter"
  @enter-cancelled="enterCancelled"
  @before-leave="beforeLeave"
  @leave="leave"
  @after-leave="afterLeave"
  @leave-cancelled="leaveCancelled"
  :css="false"
>
  <p v-if="show">hello</p>
</Transition>
```

Using the JavaScript Hooks

Declare the handlers in `<script setup>`:

```
function beforeEnter(el) {  
  el.style.opacity = 0  
}  
  
function enter(el, done) {  
  el.animate([{ opacity: 0 }, { opacity: 1 }], { duration: 300 })  
    .onfinish = done           // tell Vue the animation is over  
}
```

- With `:css="false"`, `enter` and `leave` **must** call `done()`

Custom Transition Classes

Override the generated class names to plug in a CSS animation library:

- `enter-from-class`
- `enter-active-class`
- `enter-to-class`
- `leave-from-class`
- `leave-active-class`
- `leave-to-class`

Vue 2 named the first and fourth `enter-class` and `leave-class`.

Custom Transition Classes Example

```
<link
  rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/animate.css@4.1.1">
```

```
<Transition
  enter-active-class="animate__animated animate__tada"
  leave-active-class="animate__animated animate__bounceOutRight"
>
  <p v-if="show">hello</p>
</Transition>
```

Transitions and Animations

Remove articles from the feed with an animated transition.

- Add a **Remove From Feed** icon (maybe a trash can) to each article
- When clicked, remove the article from the feed
- Wrap the list in `<TransitionGroup>` and animate articles as they leave
- Challenge: add a `-move` class so the remaining articles slide up

ES2015 (ES6) and Beyond

Objectives

- Learn what's new
- Use arrow functions and block-scoped variables
- Create generator functions
- Use classes and modules
- See how modern code is transpiled

Variable Scoping with `const` and `let`

- **`const`** creates a binding that can't be reassigned
 - The *value* isn't frozen: an object assigned to a `const` can still be changed
- **`let`** creates a **block-scoped** variable
 - `var` is scoped to the whole enclosing function
 - Redeclaring a `let` in the same scope is a syntax error
- Both are in a **temporal dead zone** until declared
 - Using one before its declaration throws a `ReferenceError`

Default to `const`; use `let` when you need to reassign. Avoid `var`.

let vs. var

var

```
var a = 5
var b = 10

if (a === 5) {
  var a = 4
  var b = 1
}
```

```
console.log(a) // 4
console.log(b) // 1
```

let

```
let a = 5
let b = 10

if (a === 5) {
  let a = 4
  let b = 1
}
```

```
console.log(a) // 5
console.log(b) // 10
```


Block-scoped Functions

ES5 (IIFEs)

```
(function () {  
  function foo() { return 1 }  
  (function () {  
    function foo() { return 2 }  
    foo() // 2  
  })()  
  foo() // 1  
})();
```

ES6 (blocks)

```
{  
  function foo() { return 1 }  
  {  
    function foo() { return 2 }  
    foo() // 2  
  }  
  foo() // 1  
}
```

Arrow Functions

Also called "fat arrow" functions: a more concise syntax for writing functions.

Function expression (ES5)

```
const increment = function (v) {  
  return v + 1  
}
```

Arrow function

```
const increment = (v) => v + 1  
  
// a block body needs return  
const inc2 = (v) => {  
  return v + 1  
}
```

Arrow Function Parameters

- Surround parameter names with parentheses:

```
(param1, param2, param3) => { statements }
```

- Parentheses are optional when there's exactly one parameter:

```
singleParam => { statements }
```

- No parameters: empty parentheses are required

```
() => { statements }
```

Arrow Functions and this

Arrow functions don't have their own `this`: they use the `this` of the surrounding code.

ES6

```
this.nums.forEach((v) => {  
  if (v % 5 === 0) {  
    this.fives.push(v)  
  }  
})
```

ES5

```
var self = this  
this.nums.forEach(function (v) {  
  if (v % 5 === 0) {  
    self.fives.push(v)  
  }  
})
```

Default Parameter Handling

ES6

```
function myFunc(x, y = 0, z = 13) {  
  return x + y + z  
}
```

```
myFunc(4) // 17
```

ES5

```
function myFunc(x, y, z) {  
  if (y === undefined) y = 0  
  if (z === undefined) z = 13  
  return x + y + z  
}
```

Rest Parameter

Gathers the remaining arguments into a single array parameter.

```
function myFunc(x, y, ...a) {  
  return (x + y) * a.length  
}
```

```
console.log(myFunc(1, 2, 'hello', true, 7)) // 9
```

- Only the last parameter can be a rest parameter
- `a` is a real array (unlike the old `arguments` object)

Spread Operator

Spreads the elements of an iterable (like an array or string) into array literals, function arguments, or object literals.

```
const params = ['hello', true, 7]
const other = [1, 2, ...params] // [1, 2, 'hello', true, 7]

myFunc(1, 2, ...params)          // myFunc(1, 2, 'hello', true, 7)

const chars = [...'foo']         // ['f', 'o', 'o']

// copy an object, overriding one property
const copy = { ...user, name: 'Ada' }
```

Template Literals

String interpolation with backticks:

```
const customer = { name: 'Penny' }  
const order = { price: 4, product: 'parts', quantity: 6 }  
  
const message = `Hi, ${customer.name}. Thank you for your order  
of ${order.quantity} ${order.product} at ${order.price}.`
```

- Any expression can go inside `${ }`
- Template literals can span multiple lines

Template Literals: Tags and Raw Strings

A **tag function** receives the string parts and the values separately, and can read the **raw** text without escape processing.

```
function tag(strings, ...values) {  
  console.log(strings.raw[0])  
  // "string text line 1 \n string text line 2"  
}  
  
tag`string text line 1 \n string text line 2`  
  
String.raw`C:\new\folder`    // "C:\new\folder" – backslashes kept
```

Enhanced Object Properties

Property shorthand: a shorter syntax for properties whose name matches a variable.

ES5

```
const obj = { x: x, y: y }
```

ES6

```
const obj = { x, y }
```

Computed Property Names

Use an expression as a property name in an object literal:

```
const obj = {  
  customer: 'Nigel',  
  ['order' + getOrderNum()]: 10,  
}  
// { customer: 'Nigel', order42: 10 }
```

- Common with dynamic keys: `{ [field]: value }`

Method Notation in Object Literals

ES6

```
const obj = {  
  foo(a, b) {  
    // ...  
  },  
  bar(x, y) {  
    // ...  
  },  
}
```

ES5

```
var obj = {  
  foo: function (a, b) {  
    // ...  
  },  
  bar: function (x, y) {  
    // ...  
  },  
}
```

Array Destructuring

Unpack array elements into variables during assignment:

```
const list = [1, 2, 3]

const [a, , b] = list      // a = 1, b = 3 (skip the middle)

;[b, a] = [a, b]           // swap without a temporary variable

const [first, ...rest] = list  // first = 1, rest = [2, 3]
```

Object Destructuring

Unpack object properties into variables during assignment:

```
const { a, b, c } = { a: 1, b: 2, c: 3 }  
console.log(a, b, c)    // 1 2 3  
  
const { title, author: { username } } = article    // nested  
const { data } = await ApiService.query('tags')    // from a response  
const { count = 0 } = options                      // with a default
```

Symbol Primitive

- A **unique** and **immutable** primitive value
- Can be used as an object property key that won't collide with any other

```
const sym1 = Symbol()  
const sym2 = Symbol('foo')  
const sym3 = Symbol('foo')
```

```
sym2 === sym3    // false – every Symbol() is unique
```

- **Well-known symbols** customize built-in behavior, e.g. `Symbol.iterator`

User-defined Iterators

- To be iterable, an object (or its prototype chain) must have a method with the key `Symbol.iterator`

```
const myIterable = {  
  *[Symbol.iterator]() {  
    yield 1  
    yield 2  
    yield 3  
  },  
}  
  
[...myIterable]    // [1, 2, 3]
```

- Iterables work with `for...of`, spread and destructuring

The for...of Statement

Iterate over all the values of an iterable object:

```
const fibonacci = {
  [Symbol.iterator]() {
    let pre = 0, cur = 1
    return {
      next() {
        ;[pre, cur] = [cur, pre + cur]
        return { done: false, value: cur }
      },
    }
  },
}

for (const n of fibonacci) {
  if (n > 1000) break
  console.log(n)
}
```

Creating and Consuming Generator Functions

A **generator** is a special kind of function that works as a factory for iterators.

```
function* idMaker() {  
  let index = 0  
  while (true) {  
    yield index++  
  }  
}  
  
const gen = idMaker()  
console.log(gen.next().value) // 0  
console.log(gen.next().value) // 1
```

Class Definition

- ES6 introduced OOP-style classes
 - Syntax on top of JavaScript's prototype-based inheritance
- Created with a **class declaration** or a **class expression**
- Can have a **constructor**, methods, getters and setters, **static** members, and (ES2022) fields and **#private** fields

Class Declaration

```
class Square {  
  constructor(height, width) {  
    this.height = height  
    this.width = width  
  }  
  
  get area() {  
    return this.height * this.width  
  }  
}  
  
const sq = new Square(3, 3)  
console.log(sq.area)    // 9
```

Class Expressions

Unnamed

```
const Square = class {  
  constructor(height, width) {  
    this.height = height  
    this.width = width  
  }  
}
```

Named

```
const Square = class Rectangle {  
  constructor(height, width) {  
    this.height = height  
    this.width = width  
  }  
}  
  
// the name Rectangle is only  
// visible inside the class body
```

Class Inheritance

```
class Rectangle extends Shape {  
  constructor(id, x, y, width, height) {  
    super(id, x, y)  
    this.width = width  
    this.height = height  
  }  
}
```

```
class Circle extends Shape {  
  constructor(id, x, y, radius) {  
    super(id, x, y)  
    this.radius = radius  
  }  
}
```

- Call `super()` before using `this` in a subclass constructor

Beyond ES2015

- **ES2016 (ES7)**
 - Exponentiation operator: `2 ** 10`
 - `Array.prototype.includes()`
- **ES2017 (ES8)**
 - `async / await`
 - `Object.entries()`, `Object.values()`
- **ES2018+** — object spread, optional chaining, `??`, top-level `await`... (see slide 8)
- **ES.Next** — the name for whatever is coming next

APPENDIX · JAVASCRIPT REVIEW

Advanced JavaScript Topics

Strict mode, this, map, and promises

"use strict"

- Introduced with ES5
- Opts in to a restricted, safer variant of JavaScript
 - Turns silent mistakes into errors (assigning to an undeclared variable)
 - `this` is `undefined` in plain function calls, not `window`
- To invoke, put `'use strict'` before any other statements in a script or function

```
function myStrictFunction() {  
  'use strict'  
  // ...  
}
```

- **Implied** in ES modules and classes, so it's redundant in modern code

Understanding this

- `this` lets a function be reused with different context objects
- It answers: which object is this function working on right now?
- Its value is set by **how the function is called**, not where it's defined
 - ...except in arrow functions, which use the surrounding `this`

The 4 Rules of this

1. **Implicit binding** — `obj.method()`: `this` is `obj`
2. **Explicit binding** — `fn.call(obj)`, `fn.apply(obj)`, `fn.bind(obj)`
3. **new binding** — `new Fn()`: `this` is the new object
4. **Default (window) binding** — a plain call `fn()`: `window` in sloppy mode, `undefined` in strict mode

Arrow functions ignore all four and use the enclosing `this`.

What is this?

- Ask: **how** was the function invoked?
- We don't know what **this** is until the function is called

```
const logName = function () {  
  console.log(this.name)  
}
```

```
author.logName = logName  
author.logName()    // this === author
```

```
logName()           // this === undefined (strict) or window
```

Implicit Binding

this refers to the object to the **left of the dot**.

```
const author = {  
  name: 'Chris',  
  homeTown: 'Detroit',  
  logName: function () {  
    console.log(this.name)  
  },  
}  
  
author.logName()    // 'Chris'
```

Explicit Binding

`.call`, `.apply` and `.bind` set `this` explicitly.

```
const logName = function () {  
  console.log(this.name)  
}  
  
const author = {  
  name: 'Chris',  
  homeTown: 'Detroit',  
}  
  
logName.call(author)    // 'Chris'
```

Explicit Binding with .call

Calls a function with a given **this** value and arguments given **individually**.

```
const logName = function (lang1) {  
  console.log(this.fname + ' ' + this.lname + ' ' + lang1)  
}  
  
const author = { fname: 'Chris', lname: 'Minnick' }  
const languages = ['JavaScript', 'HTML', 'CSS']  
  
logName.call(author, languages[0])  // 'Chris Minnick JavaScript'
```

Explicit Binding with .apply

Calls a function with a given **this** value and arguments given as an **array**.

```
const logName = function (food1, food2, food3) {  
  console.log(this.fname + ' ' + this.lname)  
  console.log(food1, food2, food3)  
}  
  
const author = { fname: 'Chris', lname: 'Minnick' }  
const favoriteFoods = ['Tacos', 'Soup', 'Sushi']  
  
logName.apply(author, favoriteFoods)
```

- Today, `logName.call(author, ...favoriteFoods)` does the same thing

Explicit Binding with .bind

Works like `.call`, but **returns a new function** instead of calling it immediately.

```
const logName = function (food) {  
  console.log(  
    `${this.fname} ${this.lname}'s favorite food was ${food}`  
  )  
}  
  
const person = { fname: 'George', lname: 'Washington' }  
  
const logMe = logName.bind(person, 'Tacos')  
  
logMe()    // "George Washington's favorite food was Tacos"
```

new Binding

When a function is called with **new**, **this** is bound to the **new object** being created.

```
const City = function (lat, long, state, pop) {  
  this.lat = lat  
  this.long = long  
  this.state = state  
  this.pop = pop  
}  
  
const sacramento = new City(38.58, -121.49, 'CA', 480000)  
console.log(sacramento.state) // 'CA'
```

Default (window) Binding

When no object is specified or implied, **this** falls back to the default:

- the global object (**window**) in non-strict scripts
- **undefined** in strict mode, modules and classes

```
const logName = function () {  
  console.log(this.name)  
}
```

// in a classic, non-module <script>:

```
window.name = 'Harry'  
logName()    // 'Harry'
```

// in a module or strict code:

```
logName()    // TypeError: Cannot read properties of undefined
```

Array.map()

- Creates a **new array** with the result of calling a function on every element

```
const newArray = arr.map((currentValue, index, array) => {  
  // return the new element  
})  
  
const prices = [10, 20, 30]  
const withTax = prices.map((p) => p * 1.1)    // [11, 22, 33]
```

- The callback receives the current value, its index, and the array
- The original array isn't changed

Promises

Asynchronous JavaScript, from callbacks to async/await

What Are Promises?

- An abstraction for asynchronous programming; an alternative to callbacks
- A promise represents the **eventual result** of an async operation
- It's always in one of three states:
 - **pending** — the initial state
 - **fulfilled** — the operation succeeded, with a value
 - **rejected** — the operation failed, with a reason (an error)
- Once fulfilled or rejected, it's **settled** and never changes

Promises vs. Event Listeners

- Event listeners are good for things that can happen **many times** to one object (clicks, keypresses)
- A promise can only succeed or fail **once**
- If a promise has already settled, you can still react to it at any time

```
readJSON(filename).then(onSuccess, onFailure)
```

Why Use Promises?

- **Chain** them to transform values or run more async steps in order
- **Cleaner code**: one `.catch()` handles errors for the whole chain
- Avoid the problems of deeply nested callbacks:
 - "callback hell", the "pyramid of doom", the "Christmas tree"
- **Combine** them: `Promise.all`, `Promise.allSettled`, `Promise.race`, `Promise.any`
- They're the foundation of `async / await`

Callback vs. Promise

Files: demos/callback-vs-promise

Callback

```
import fs from 'node:fs'
fs.readFile('text.txt', (err, file) => {
  if (err) console.error(err)
  else console.log(file.toString())
})
```

Promise

```
import { readFile } from 'node:fs/promises'
readFile('text.txt')
  .then((data) => console.log(data.toString()))
  .catch(console.error)
```

Creating Promises

Wrap a callback-based API in a **Promise**:

```
import fs from 'node:fs'

function readFileSync(file, encoding) {
  return new Promise((resolve, reject) => {
    fs.readFile(file, encoding, (err, data) => {
      if (err) return reject(err)
      resolve(data)
    })
  })
}

readFileSync('text.txt', 'utf8').then(console.log, console.error)
```

- In Node, **util.promisify()** does this for you

Async / Await

async functions always return a promise; **await** pauses until a promise settles.

```
async function f() {  
  return 1  
}  
f().then(console.log) // 1  
  
async function getUser() {  
  const response = await fetch('/api/user.json')  
  if (!response.ok) throw new Error(`HTTP ${response.status}`)  
  return response.json()  
}  
  
try {  
  const user = await getUser()  
} catch (err) {  
  console.error(err)  
}
```

Disclaimer and Copyright

Disclaimer

WatzThis? takes care to ensure the accuracy and quality of this courseware and related courseware files. We cannot guarantee the accuracy of these materials. The courseware and related files are provided without any warranty whatsoever, including but not limited to implied warranties of merchantability or fitness for a particular purpose. Use of screenshots, product names and icons in the courseware are for editorial purposes only. No such use should be construed to imply sponsorship or endorsement of the courseware, nor any affiliation of such entity with WatzThis?.

Third-Party Information

This courseware contains links to third-party web sites that are not under our control and we are not responsible for the content of any linked sites. If you access a third-party web site mentioned in this courseware, then you do so at your own risk. We provide these links only as a convenience, and the inclusion of the link does not imply that we endorse or accept responsibility for the content on those third-party web sites. Information in this courseware may change without notice and does not represent a commitment on the part of the authors and publishers.

Copyright

Copyright 2026, WatzThis?. All rights reserved. Screenshots used for illustrative purposes are the property of the software proprietor. This publication, or any part thereof, may not be reproduced or transmitted in any form by any means, electronic or mechanical, including photocopying, recording, storage in an information retrieval system, or otherwise, without express written permission of WatzThis, 567 Commercial St, Astoria, OR 97103. www.watzthis.com

Help us improve our courseware

Please send your comments and suggestions by email to info@watzthis.com